Árboles rojinegros y el contenedor `set`

Colores, rotaciones y lo que hay dentro de `map`

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Explicar por qué un ABB que recibe claves en orden pierde su costo logarítmico y cuánto costaría reconstruirlo tras cada inserción (OA1).
- Decidir si un árbol coloreado cumple las cinco reglas de un árbol rojinegro y explicar en palabras por qué su altura queda por debajo de $2\log_2(n+1)$ (OA2).
- Seguir a mano una rotación y una inserción en un árbol rojinegro, con el caso que corresponde en cada paso (OA2).

Objetivos de aprendizaje II

Al finalizar esta sesión, el estudiante podrá también

- Usar `set` con `insert`, `erase`, `find`, `count`, `lower_bound` y `upper_bound`, y dar el costo de cada operación (OA1, OA3).
- Escoger entre `set`, `map` y `unordered_map` según lo que pide el problema y resolver problemas de juez con `set` (OA5).

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

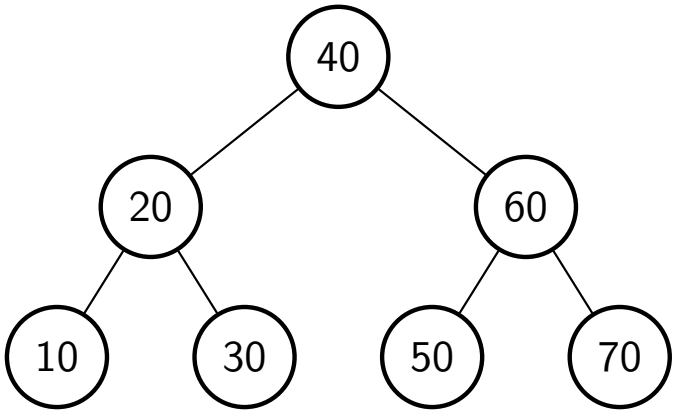
6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Lo que quedó del árbol binario de búsqueda I



- En cada nodo, las claves de la izquierda son menores y las de la derecha, mayores.
- Buscar, insertar y borrar bajan por un solo camino: cuestan $O(h)$.
- Todo árbol binario con n nodos cumple $\lfloor \log_2 n \rfloor \leq h \leq n - 1$.
- El inorden sale ordenado: 10 20 30 40 50 60 70.

La forma la decide el orden de llegada

Estas siete claves insertadas como 40 20 60 10 30 50 70 dan altura 2. Insertadas como 10 20 30 40 50 60 70 dan una lista de altura 6.

Números de pedido I

Problema

Una tienda en línea numera sus pedidos con un contador: cada pedido nuevo recibe un número mayor que todos los anteriores. Los números se guardan en un ABB para responder si un pedido existe, para anularlo y para listar los pendientes de menor a mayor. Llegan cientos de miles de pedidos al día.

¿Entendimos el problema? I

Entradas

Números de pedido que llegan en orden creciente: `10 20 30 40 50 60 70 80`, y preguntas sobre ellos.

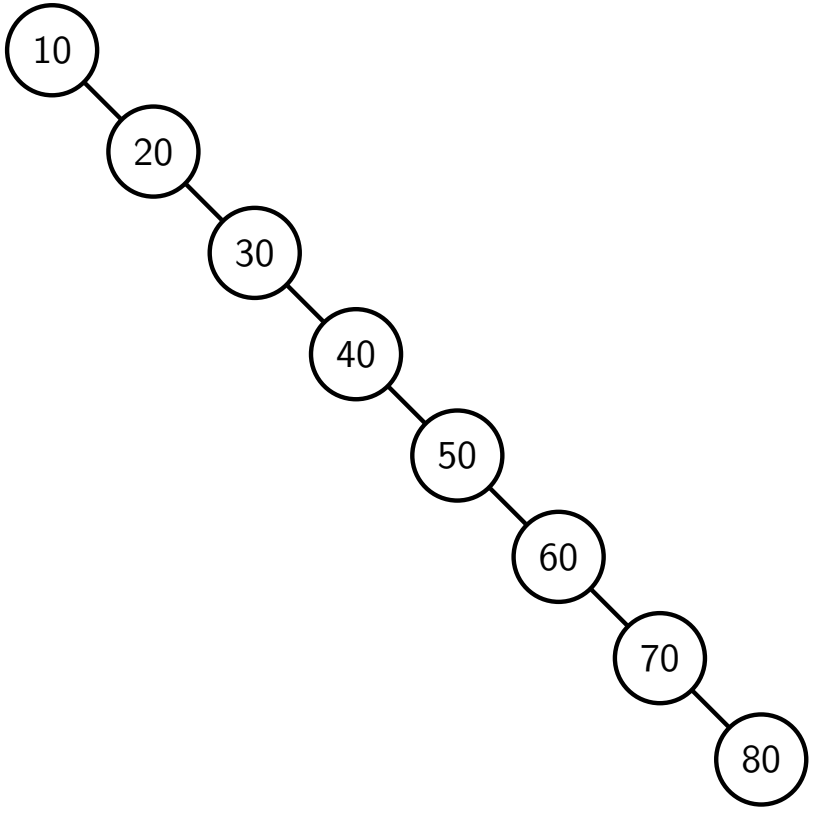
Salidas

`si` o `no` para cada pregunta, y la lista en orden cuando se pide.

Pregunta

¿Qué árbol queda tras insertar los ocho números? ¿Cuántas comparaciones cuesta preguntar por el pedido 80?

¿Entendimos el problema? II



Respuesta

Cada número es mayor que todos los que ya están, así que siempre baja por la derecha hasta el fondo. Queda una lista de altura 7, y preguntar por el 80 compara 8 veces.

Con n pedidos, el i-ésimo compara $i - 1$ veces al entrar, y en total son $\frac{n(n-1)}{2} = \Theta(n^2)$ comparaciones.

¿Entendimos el problema? III

`medir_set.cpp` inserta `1, 2, ..., n` en el ABB de `abb.h`, con `insertarIterativa`, y mide la altura y el tiempo de procesador (sin optimizar, en el equipo donde se preparó la sesión):

n	altura del ABB	segundos
1 000	999	0,0008
2 000	1 999	0,0034
4 000	3 999	0,0128
8 000	7 999	0,0512
16 000	15 999	0,2021
32 000	31 999	0,7989

Cada vez que n se dobla, el tiempo se multiplica por cerca de 4: es el $\Theta(n^2)$. A ese ritmo, un día con 500 000 pedidos tomaría más de tres minutos solo en insertar.

Primera idea: reconstruir el árbol balanceado I

Pregunta

`construirBalanceado(a, ini, fin)` arma el ABB de menor altura a partir de un arreglo ordenado. ¿Qué pasa si, después de cada inserción, se saca el inorden a un arreglo y se reconstruye el árbol con esa función?

Primera idea: reconstruir el árbol balanceado II

Respuesta

El árbol queda siempre con altura $\lfloor \log_2 n \rfloor$, y buscar cuesta $\Theta(\log n)$. Pero cada inserción paga el inorden, $\Theta(n)$, y la reconstrucción, otra vez $\Theta(n)$.

	Una inserción	n inserciones
ABB con claves en orden	$\Theta(n)$	$\Theta(n^2)$
ABB reconstruido cada vez	$\Theta(n)$	$\Theta(n^2)$
Lo que se quiere	$O(\log n)$	$O(n \log n)$

Se arregló la búsqueda y la inserción quedó igual

Reconstruir mueve los n nodos para acomodar uno solo. La altura mínima exacta es una exigencia demasiado cara.

Primera idea: reconstruir el árbol balanceado III

Segunda idea

No exigir la altura mínima: aceptar un margen y repararlo con cambios *locales*, cerca del nodo que entró, sin tocar el resto del árbol.

- El margen: ningún camino de la raíz hacia abajo puede ser más del doble de largo que otro.
- La reparación: cambiar colores y girar unos pocos nodos, cada cosa en $O(1)$, sobre el camino por donde bajó la inserción.

El árbol rojinegro (CLRS, cap. 13; Thareja, §10.5) se construye sobre esa idea.

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Definición (CLRS, §13.1; Thareja, §10.5.1) I

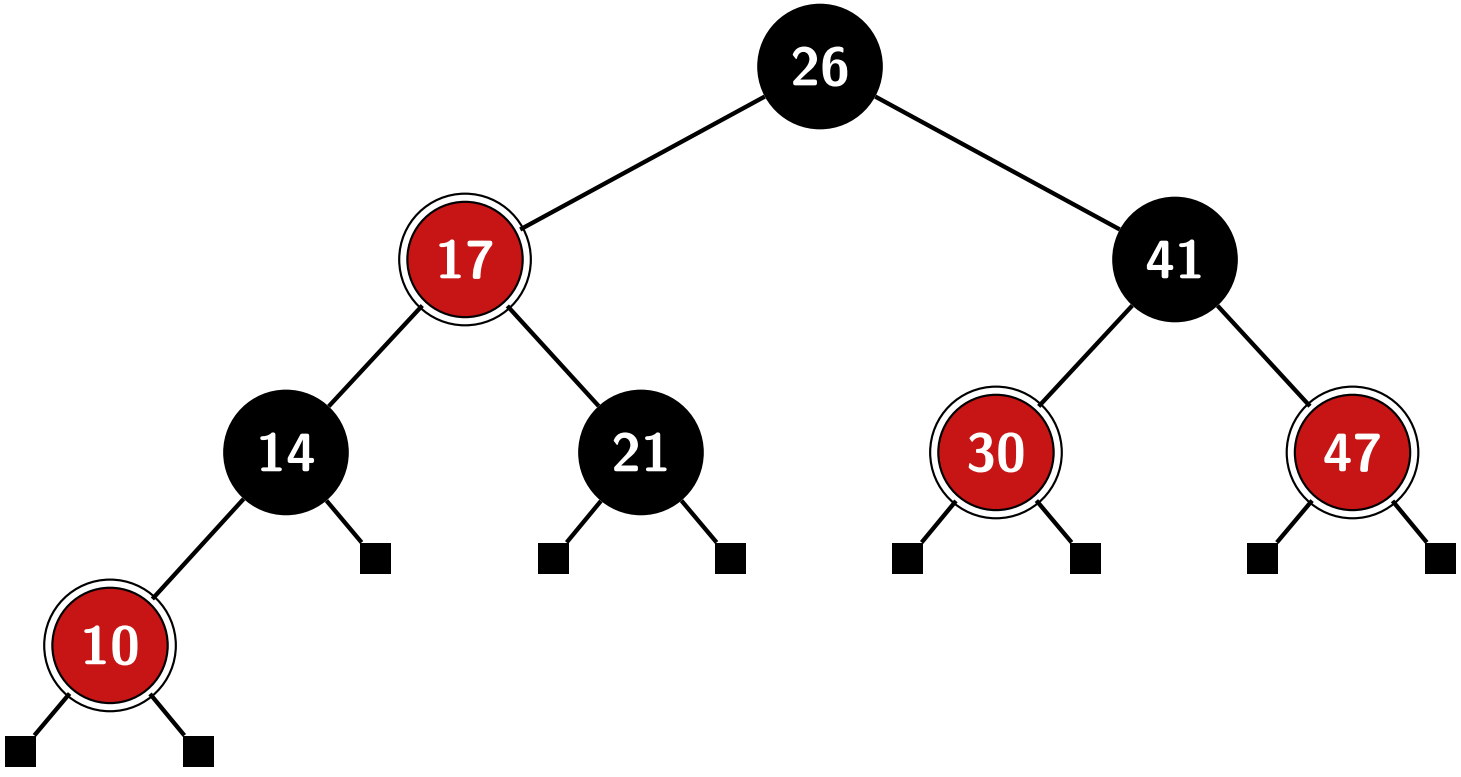
Árbol rojinegro

Un ABB donde cada nodo lleva un color, rojo o negro, y que cumple:

1. Todo nodo es rojo o negro.
2. La raíz es negra.
3. Toda hoja (`NIL`) es negra.
4. Si un nodo es rojo, sus dos hijos son negros.
5. Para cada nodo, todos los caminos desde él hasta las hojas `NIL` de abajo tienen la misma cantidad de nodos negros.

Las hojas `NIL` son los punteros `NULL` del ABB, vistos como hojas negras sin clave. Las claves viven en los nodos internos.

Definición (CLRS, §13.1; Thareja, §10.5.1) II

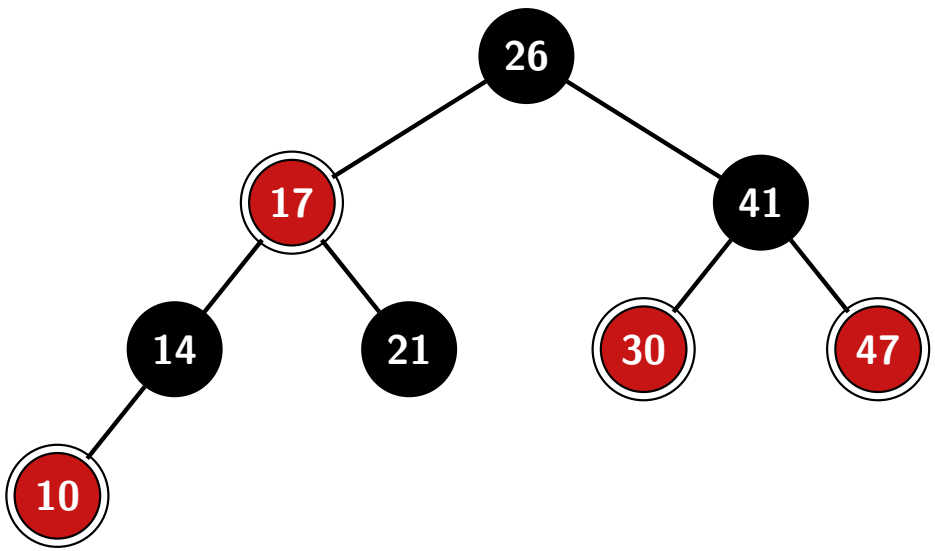


Negro: relleno negro. Rojo: relleno rojo con doble borde. Los cuadritos son las hojas `NIL`. Ningún rojo tiene un hijo rojo, y desde la raíz todo camino hasta un `NIL` pasa por 2 negros sin contar la raíz.

Altura negra I

Definición (CLRS, §13.1)

La *altura negra* de un nodo x, $bh(x)$, es la cantidad de nodos negros en un camino desde x, sin contar x, hasta una hoja `NIL`, contándola. Por la regla 5 no depende del camino.



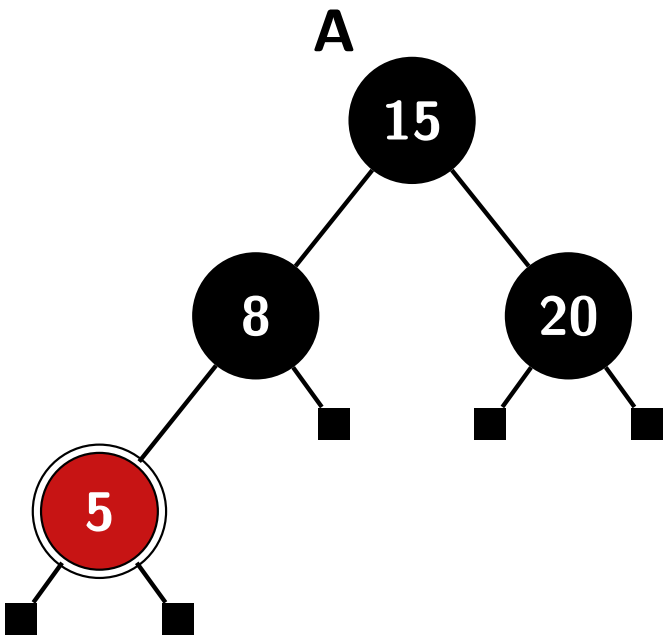
Nodo	Un camino	*bh*
26	17, 14, 10, `NIL`	2
26	41, 47, `NIL`	2
17	21, `NIL`	2
41	30, `NIL`	1
14	10, `NIL`	1
10	`NIL`	1

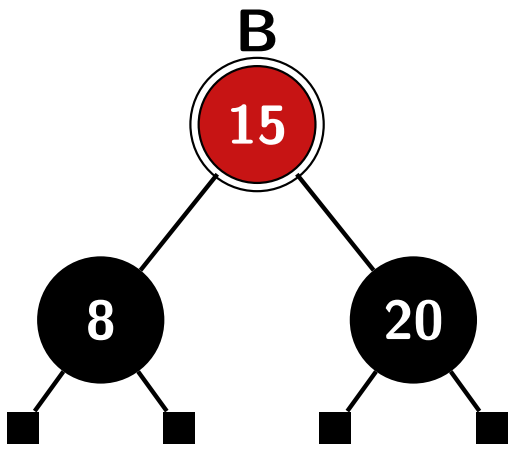
En cada fila se cuentan solo los negros del camino: el 17, el 10, el 47 y el 30 son rojos y no suman.

¿Cuáles cumplen? I

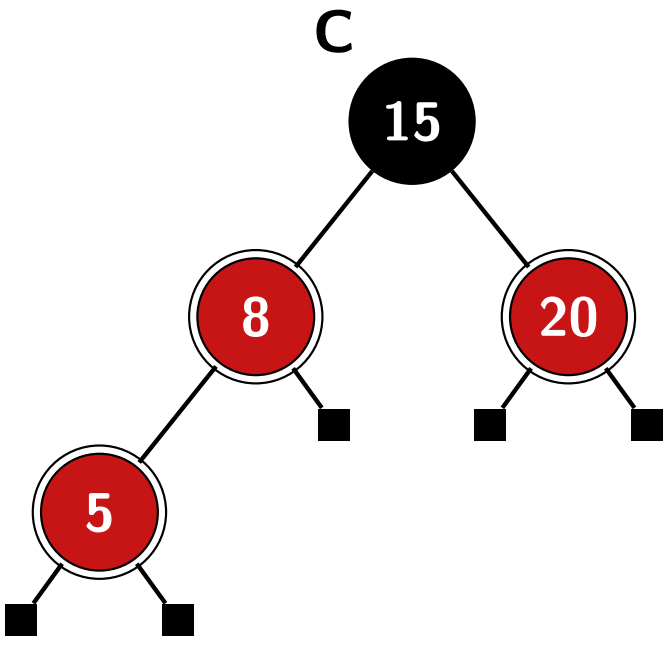
Pregunta

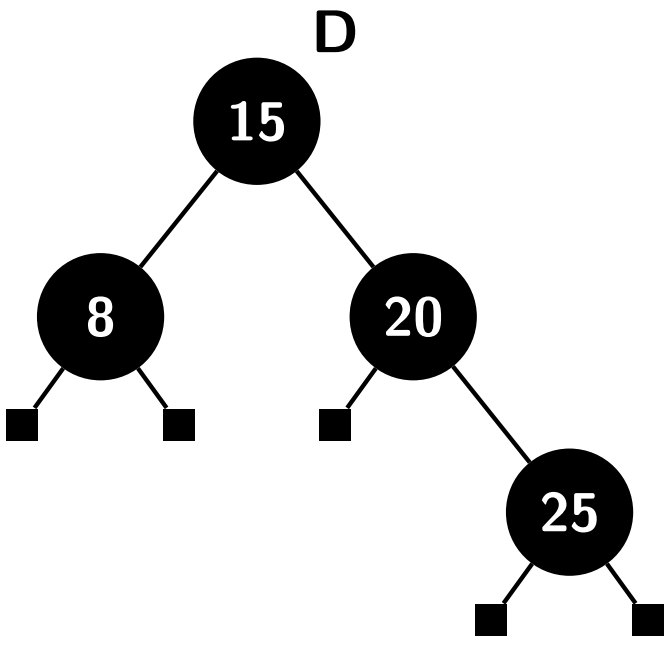
Para cada árbol, decidan si es rojinegro. Si no lo es, ¿qué regla rompe?





¿Cuáles cumplen? II





¿Cuáles cumplen? III

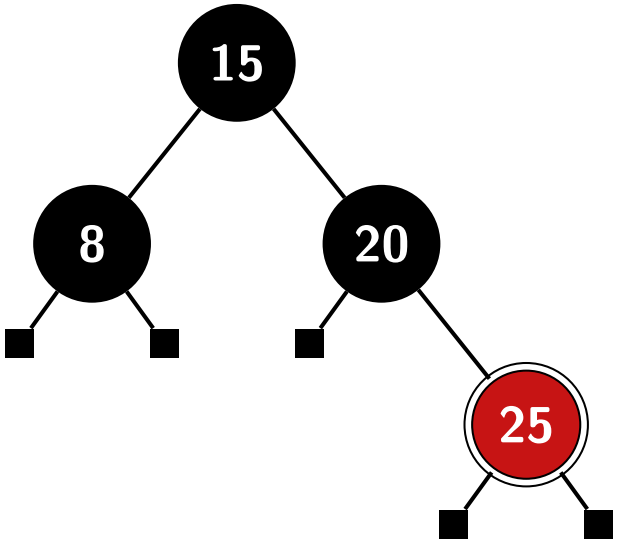
Respuesta

- **A** cumple. Desde el 15, todo camino pasa por 2 negros sin contar el 15: 8 y `NIL`, o 20 y `NIL`. El 5 es rojo y no suma.
- **B** rompe la regla 2: la raíz es roja.
- **C** rompe la regla 4: el 8 es rojo y su hijo 5 también. Las alturas negras sí coinciden: todos los caminos pasan por un solo negro, el `NIL`.
- **D** rompe la regla 5: desde el 15, el camino por el 8 pasa por 2 negros y el que baja por 20 y 25, por 3.

¿Cuáles cumplen? IV

Pregunta

D está todo en negro y no cumple. ¿Se puede pintar algún nodo de rojo para arreglarlo?



Respuesta

Sí, y de una sola forma: el 25 rojo. Ese camino pierde un negro y queda con 2, como los demás; el 25 tiene hijos `NIL`, que son negros, y la regla 4 se cumple.

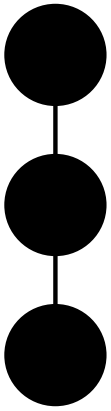
Pintar todo de negro solo funciona si todas las hojas están al mismo nivel. Los rojos son el margen que deja tener caminos de distinto largo.

Cuánto puede medir un rojinegro I

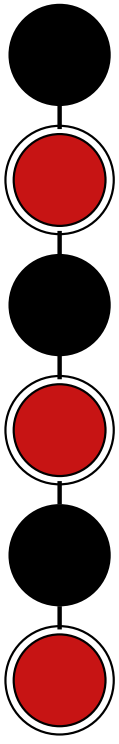
Pregunta

Si desde la raíz todo camino hasta un `NIL` pasa por b nodos negros internos, ¿cuál es el camino más corto posible? ¿Y el más largo?

Cuánto puede medir un rojinegro II



el más corto:
b nodos



el más largo:
$2b$ nodos

- El más corto es todo negro: b nodos.
- El más largo intercala un rojo después de cada negro, porque dos rojos seguidos rompen la regla 4: $2b$ nodos.
- Entonces ningún camino es más del doble de largo que otro.

Cuánto puede medir un rojinegro III

Ningún camino llega a un `NIL` antes de pasar por b nodos internos, así que los niveles $0, 1, \ldots, b-1$ están completos:

$$n \geq 1 + 2 + \cdots + 2^{b-1} = 2^b - 1 \quad \Longrightarrow \quad b \leq \log_2(n+1).$$

El camino más largo tiene a lo sumo $2b$ nodos, de modo que $h < 2b \leq 2\log_2(n+1)$. CLRS lo demuestra en el lema 13.1.

n	$\lfloor \log_2 n \rfloor$	$2\log_2(n+1)$	ABB, peor caso
15	3	8	14
1 000	9	19,9	999
1 000 000	19	39,9	999 999
10^9	29	59,8	$10^9 - 1$

Con mil millones de claves, buscar en un rojinegro baja a lo sumo 60 niveles, sin importar el orden de llegada.

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

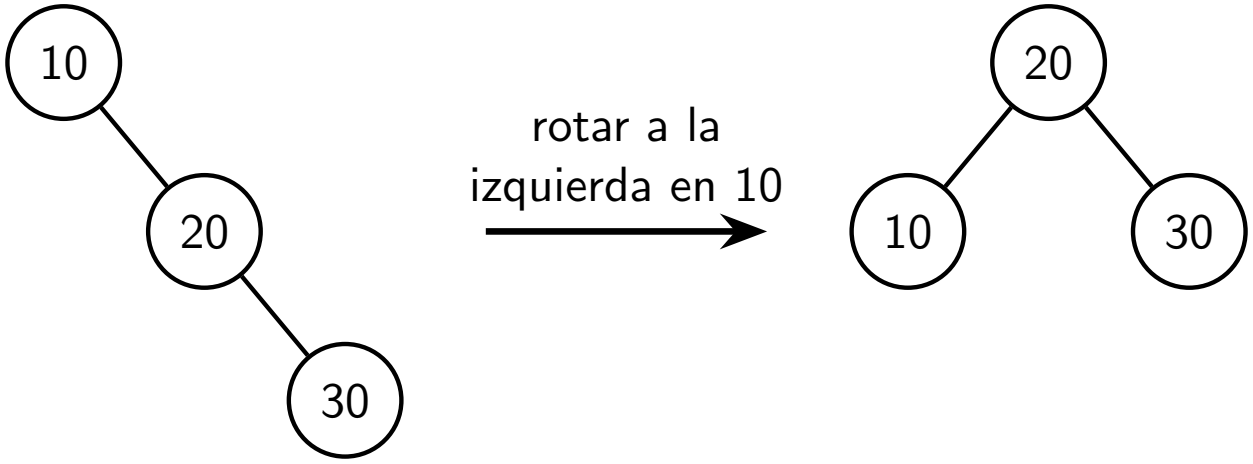
9 Cierre

Subir un nodo sin romper el orden I

Pregunta

El ABB `10 20 30` es una lista hacia la derecha. ¿Cómo se deja el 20 en la raíz, con el 10 y el 30 como hijos, cambiando solo punteros y sin crear ni borrar nodos?

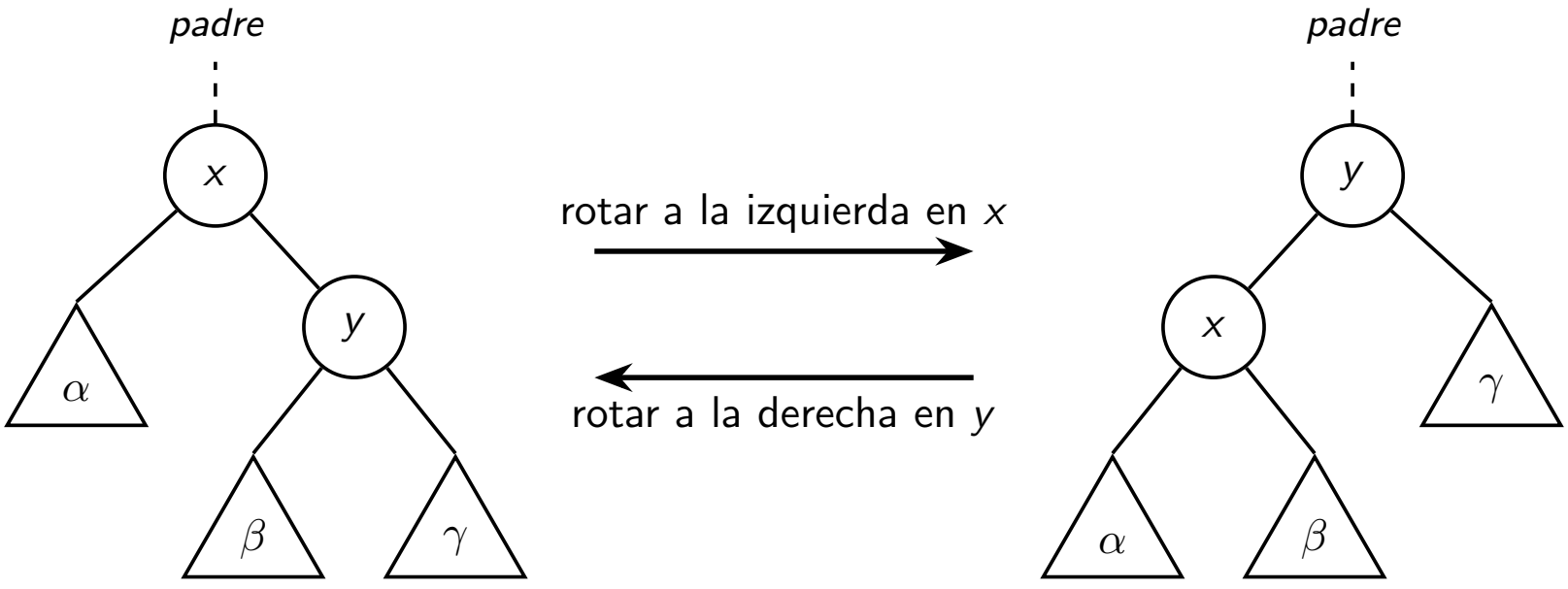
Subir un nodo sin romper el orden II



Respuesta

El 20 sube y el 10 baja a ser su hijo izquierdo. El inorden sigue siendo 10 20 30, y la altura pasó de 2 a 1. El giro se llama *rotación*.

Las dos rotaciones (CLRS, §13.2) I



α, β y γ son subárboles completos, quizás vacíos. En los dos árboles el inorden es α x β y γ: la rotación conserva el orden del ABB.

Las dos rotaciones (CLRS, §13.2) II

Lo que cambia al rotar a la izquierda en x

1. El hijo derecho de x deja de ser y y pasa a ser β.
2. El hijo izquierdo de y deja de ser β y pasa a ser x.
3. El padre de x, o la raíz si x era la raíz, apunta ahora a y.

Son tres enlaces, sin importar cuántos nodos tengan α, β y γ. Rotar cuesta $O(1)$ en tiempo y en espacio. La rotación a la derecha en y es la misma figura leída de derecha a izquierda.

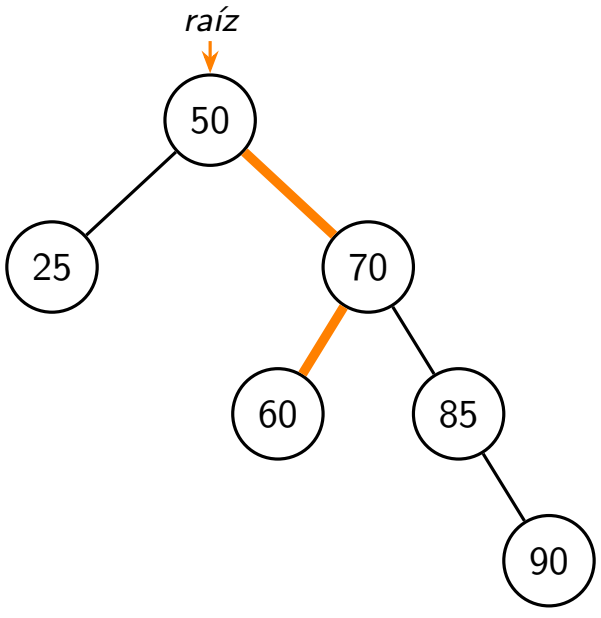
β cambia de padre

β estaba a la izquierda de y y termina a la derecha de x. Sus claves son mayores que x y menores que y, así que en las dos posiciones respeta el orden. Olvidar moverla deja un subárbol colgando de dos padres o de ninguno.

Una rotación con números I

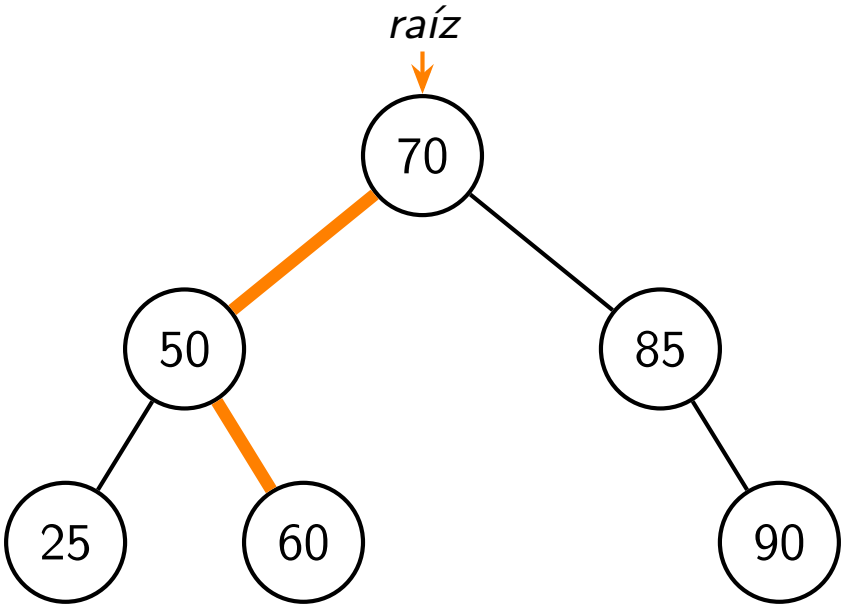
Pregunta

Roten a la izquierda en el 50. ¿Quién queda en la raíz? ¿Dónde termina el 60? ¿Cambia el inorden?



En naranja, los enlaces que van a cambiar: el hijo derecho del 50, el hijo izquierdo del 70 y el puntero a la raíz.

Una rotación con números II



Respuesta

El 70 sube a la raíz y el 50 baja a su izquierda. El 60 era el β: pasa de hijo izquierdo del 70 a hijo derecho del 50. El inorden sigue siendo 25 50 60 70 85 90, y la altura bajó de 3 a 2.

Una rotación con números III

Pregunta

¿Qué rotación devuelve el árbol a como estaba?

Respuesta

Rotar a la derecha en el 70: el 50 vuelve a la raíz y el 60 vuelve a colgar del 70. Cada rotación se deshace con la del lado contrario en el nodo que subió.

Operación	Tiempo	Espacio aparte	Por qué
rotar a la izquierda	$O(1)$	$O(1)$	cambian tres enlaces
rotar a la derecha	$O(1)$	$O(1)$	la misma figura al revés

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Insertar: como en el ABB, y de rojo I

CLRS, §13.3

1. Se baja como en el ABB y el nodo nuevo, z, se cuelga en el `NIL` donde termina la búsqueda.
2. z se pinta de rojo, con dos hojas `NIL` negras.
3. Si quedó roto algo, se arregla subiendo por el camino con recoloreos y rotaciones.

Pregunta

¿Por qué rojo y no negro? ¿Qué reglas se pueden romper al colgar un nodo rojo?

Insertar: como en el ABB, y de rojo II

Respuesta

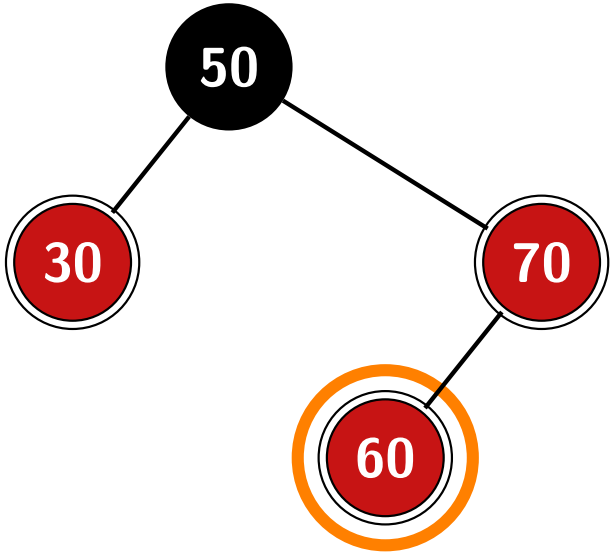
Un nodo negro nuevo le suma un negro a los caminos que pasan por él y a ningún otro: rompería la regla 5 en todos sus antepasados, y eso no se arregla mirando cerca. Un rojo no cambia ninguna altura negra. Solo puede romper dos reglas:

- la 2, si z es la raíz: se pinta de negro y listo;
- la 4, si el padre de z también es rojo.

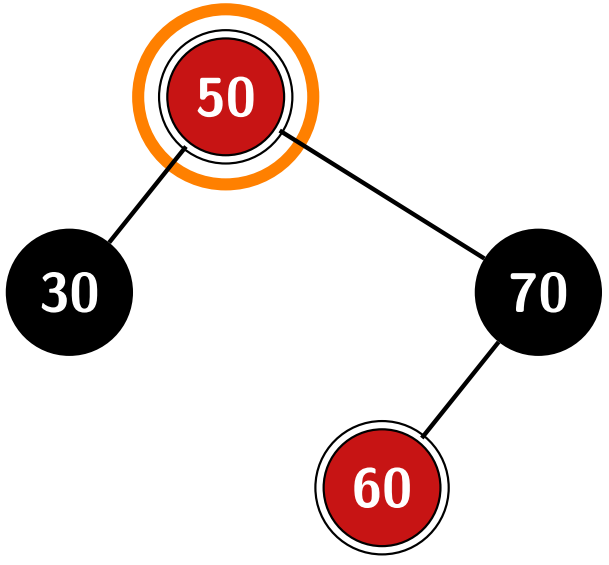
Cuando el padre es rojo, el abuelo es negro (el árbol cumplía antes), y lo que decide qué hacer es el *tío*: el otro hijo del abuelo.

Caso del tío rojo: recolorear y subir I

Se inserta el 60. Su padre 70 es rojo y el tío 30 también.

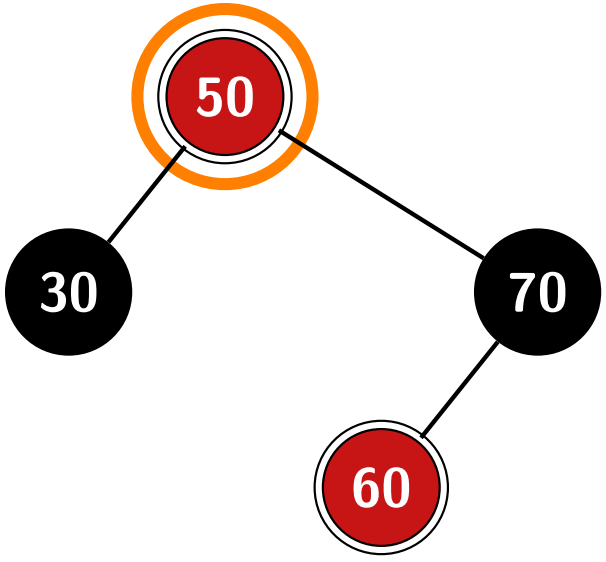


El 60 entra rojo bajo el 70, rojo: se rompe la regla 4.

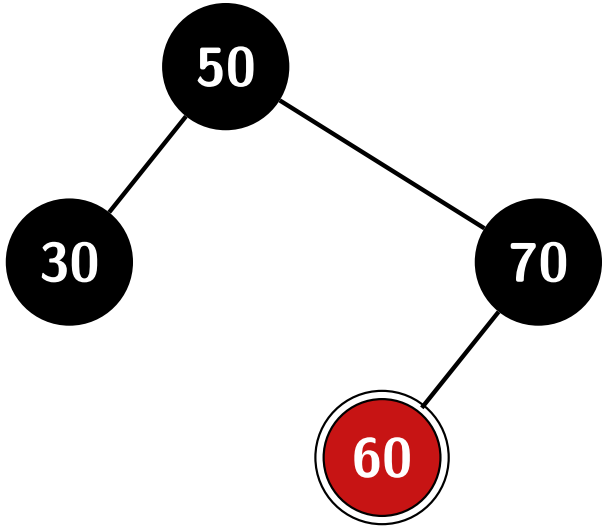


Padre y tío pasan a negro y el abuelo 50, a rojo. Las alturas negras no cambian. Ahora z es el abuelo.

Caso del tío rojo: recolorear y subir II



El 50 quedó rojo y es la raíz: se rompe la regla 2.



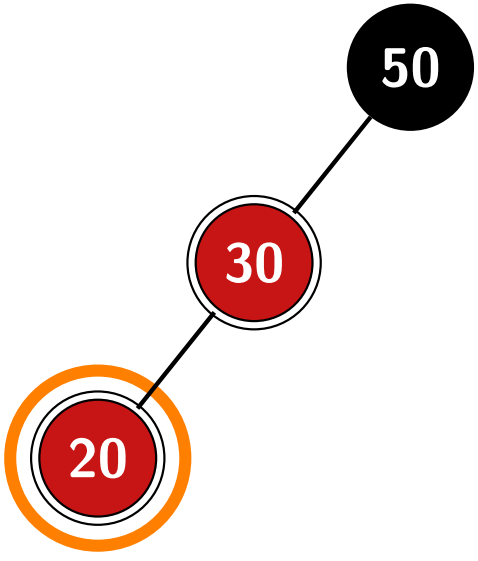
La raíz se pinta de negro. Todos los caminos ganan un negro a la vez, y la regla 5 sigue en pie.

Caso 1 de CLRS

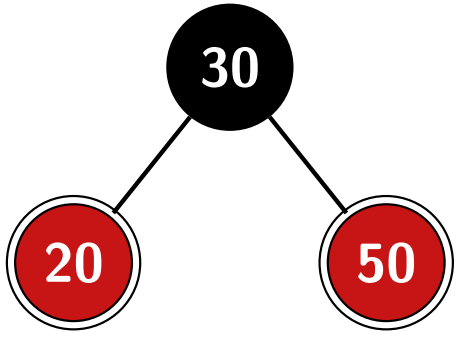
Tío rojo: padre y tío a negro, abuelo a rojo, y z sube dos niveles, al abuelo. Si el bisabuelo también es rojo, se repite arriba.

Caso del tío negro, en línea: una rotación I

Se inserta el 20 en `50 30`. Su padre 30 es rojo y el tío es `NIL`, negro. El 20, el 30 y el 50 están en línea: los dos enlaces van a la izquierda.



Recolorear no sirve: pintar el 30 de negro le suma un negro solo a los caminos de la izquierda.



El 30 pasa a negro, el 50 a rojo, y se rota a la derecha en el 50. La altura negra queda igual en los dos lados.

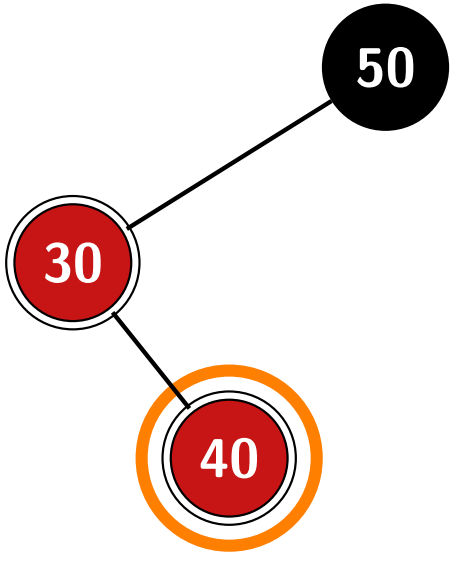
Caso del tío negro, en línea: una rotación II

Caso 3 de CLRS

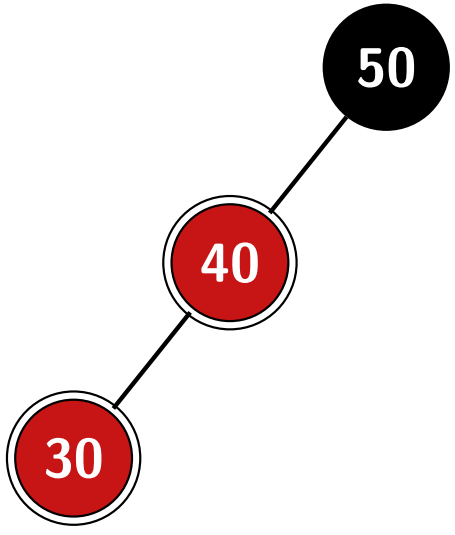
Tío negro y z en línea con su padre: el padre pasa a negro, el abuelo a rojo, y se rota en el abuelo hacia el lado contrario. Termina el arreglo.

Caso del tío negro, en ángulo: dos rotaciones I

Se inserta el 40 en `50 30`. El tío vuelve a ser `NIL`, pero ahora el 40 es hijo derecho y su padre 30 es hijo izquierdo: el camino quiebra en ángulo.

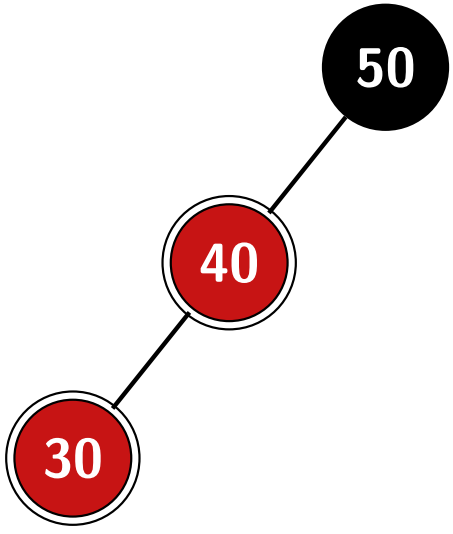


Rotar en el 50 ahora dejaría el 40 colgando del 30, otra vez rojo bajo rojo.

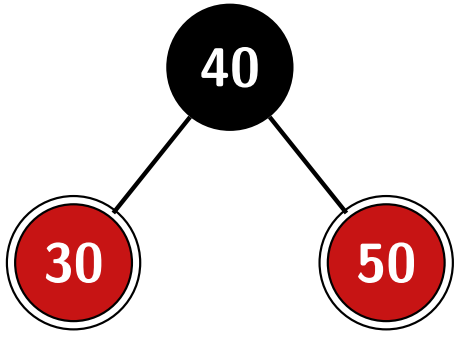


Primero se rota a la izquierda en el padre 30: quedan en línea, con el 30 abajo. Ahora z es el 30.

Caso del tío negro, en ángulo: dos rotaciones II



En línea: es el caso anterior.



El 40 pasa a negro, el 50 a rojo, y se rota a la derecha en el 50.

Caso 2 de CLRS

Tío negro y z en ángulo con su padre: se rota en el padre para ponerlos en línea, y se sigue con el caso 3. Dos rotaciones en total.

Los tres casos en una tabla I

z es rojo y su padre también; el abuelo es negro.

Situación	Qué se hace	Después
Tío rojo (caso 1)	padre y tío a negro, abuelo a rojo	z sube al abuelo; puede repetirse
Tío negro, en ángulo (caso 2)	rotar en el padre, hacia afuera	queda en línea: caso 3
Tío negro, en línea (caso 3)	padre a negro, abuelo a rojo, rotar en el abuelo	termina
z es la raíz	pintarla de negro	termina

Izquierda y derecha

Los tres casos se escribieron con el padre a la izquierda del abuelo. Si está a la derecha, todo se refleja: donde dice rotar a la derecha, se rota a la izquierda.

Traza: 10 20 30 40 50 60 70 80 I

Los mismos números de pedido que armaban la lista. El anillo naranja marca a z.



10: entra rojo y es la raíz; se rompe la regla 2.

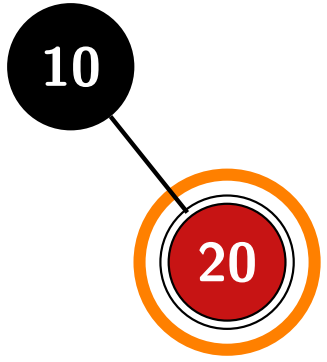


Se pinta de negro.

Traza: 10 20 30 40 50 60 70 80 II

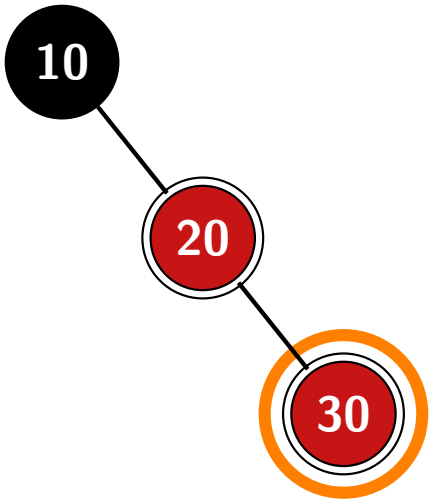


Antes.



20: entra rojo bajo el 10, negro. No se rompe nada.

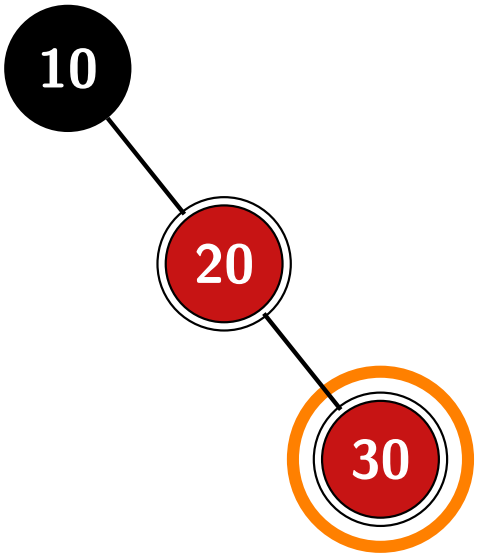
Traza: 10 20 30 40 50 60 70 80 III



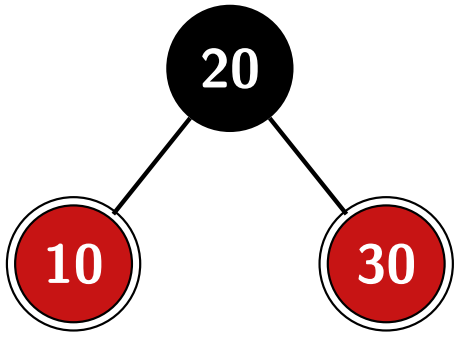
Pregunta

Entra el 30. ¿Qué regla se rompió? ¿Quién es el tío y de qué color es? ¿Qué caso es?

Traza: 10 20 30 40 50 60 70 80 IV

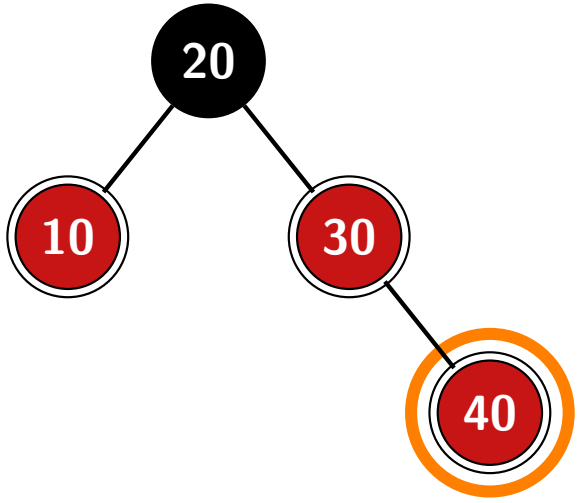


El padre 20 es rojo. El tío es el hijo izquierdo del 10, un `NIL`: negro. 10, 20 y 30 en línea a la derecha: caso 3.



El 20 pasa a negro y el 10 a rojo; se rota a la izquierda en el 10.

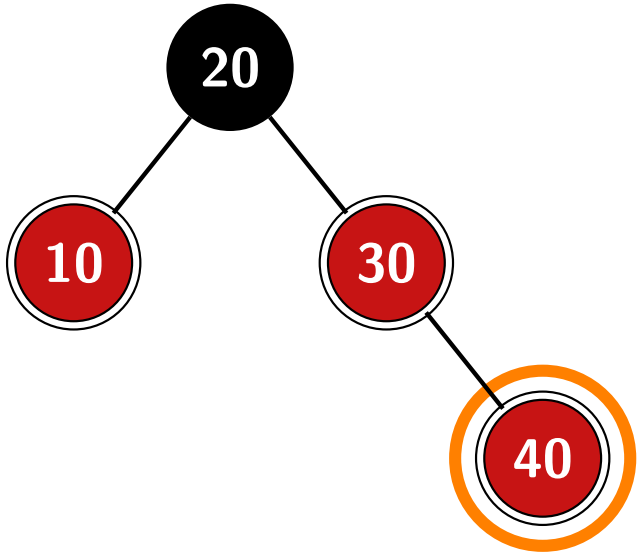
Traza: entra el 40 I



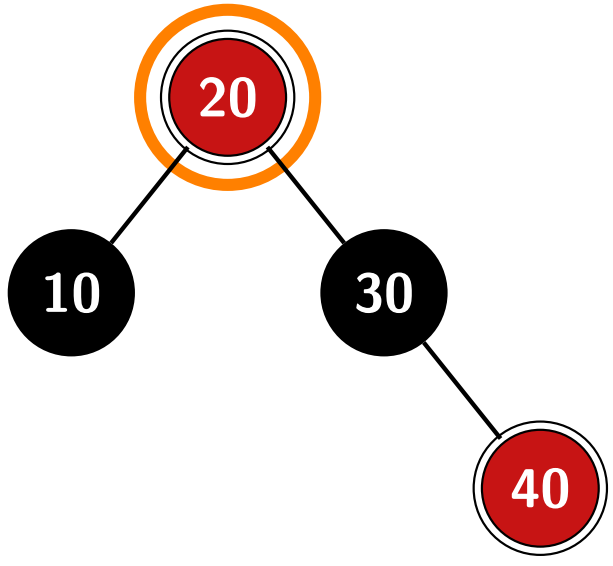
Pregunta

El padre del 40 es el 30, rojo. ¿De qué color es el tío? ¿Hace falta rotar?

Traza: entra el 40 II

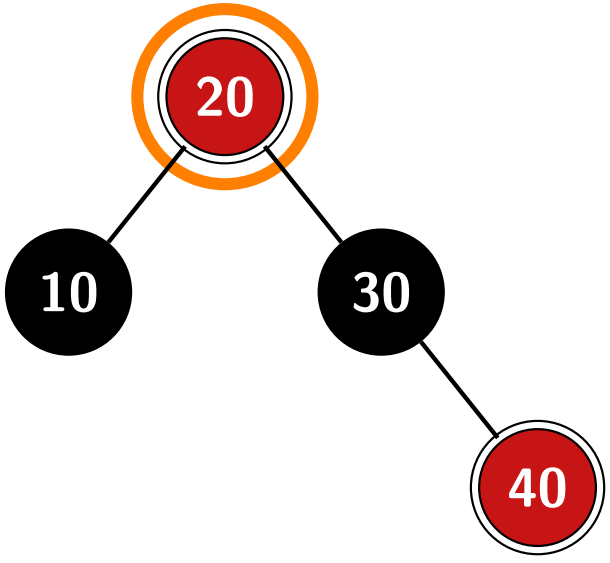


El tío es el 10, rojo: caso 1.

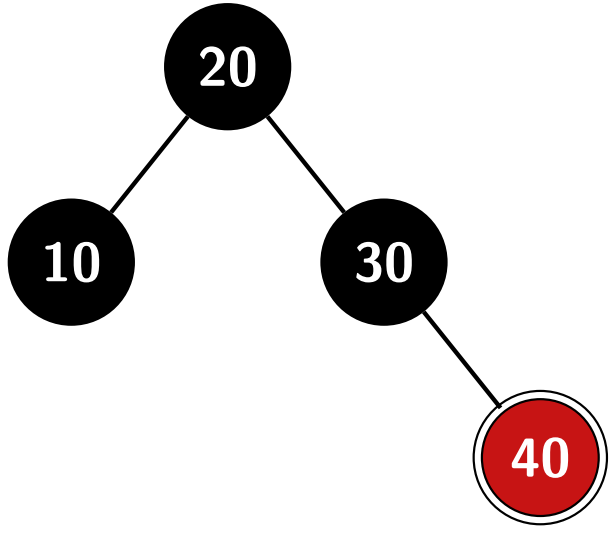


10 y 30 a negro, 20 a rojo; z sube al 20.

Traza: entra el 40 III

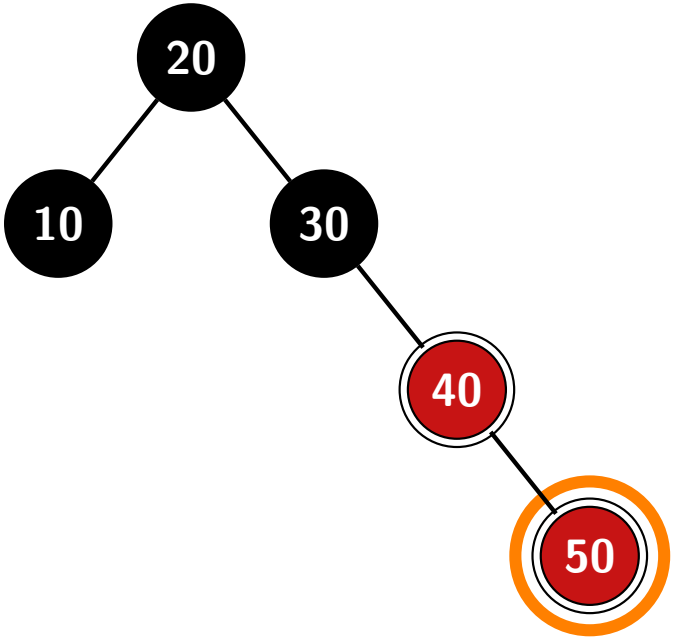


z es la raíz y es roja.

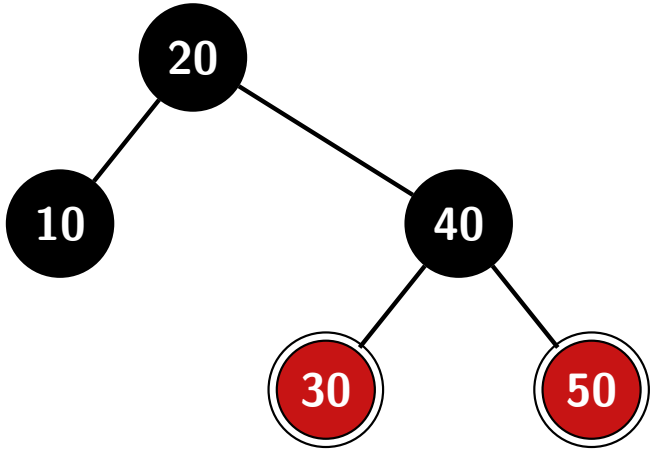


Se pinta de negro. Ninguna rotación.

Traza: entran el 50 y el 60 I

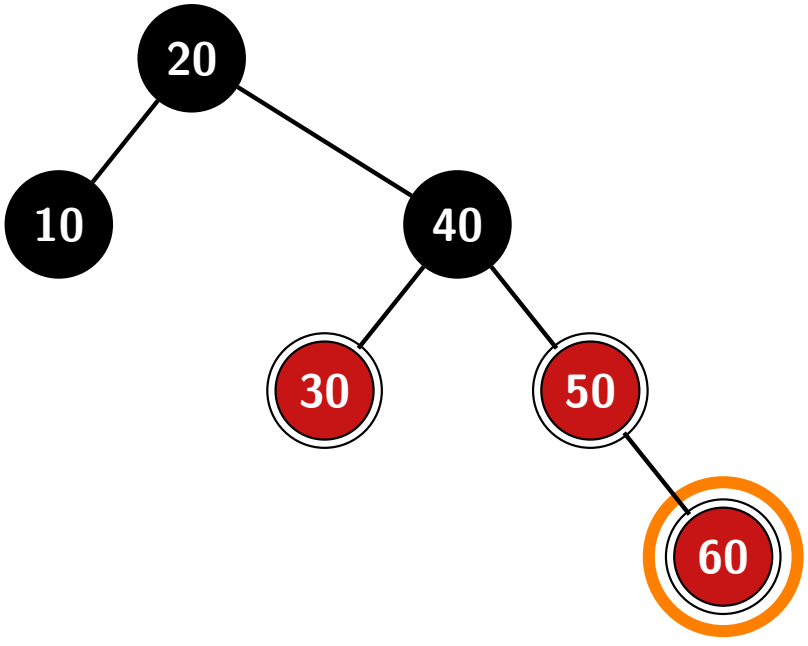


50: padre 40 rojo, tío `NIL` negro (el hijo izquierdo del 30), en línea a la derecha: caso 3.

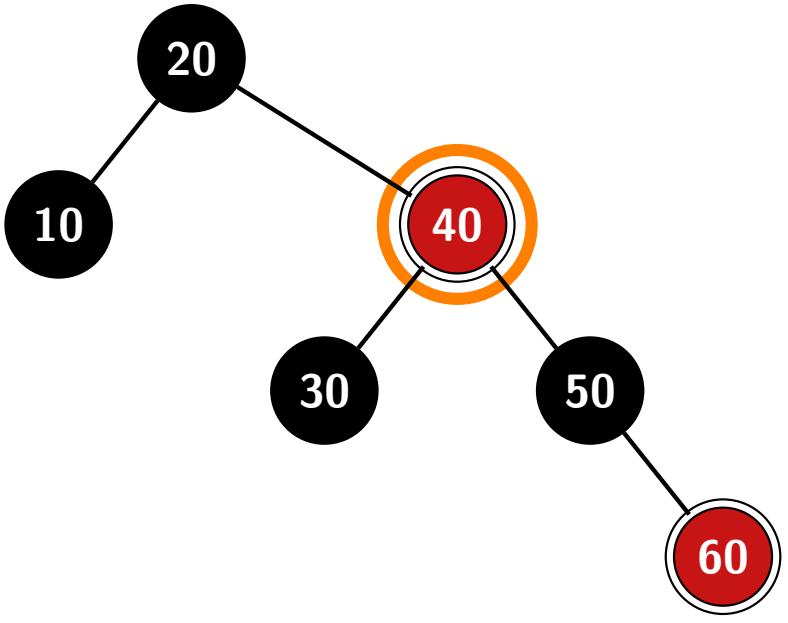


40 a negro, 30 a rojo, rotar a la izquierda en el 30.

Traza: entran el 50 y el 60 II

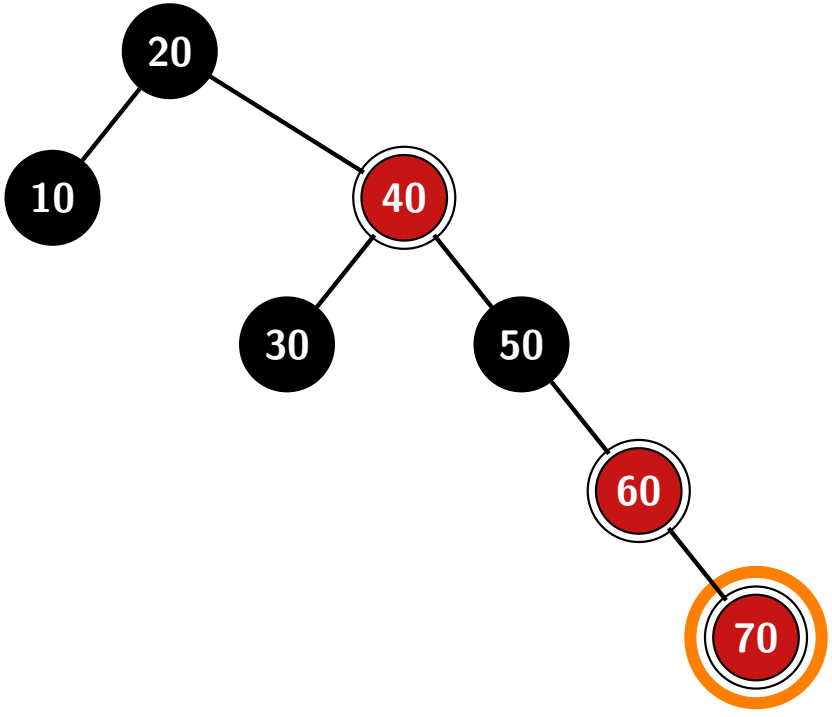


60: padre 50 rojo, tío 30 rojo: caso 1.

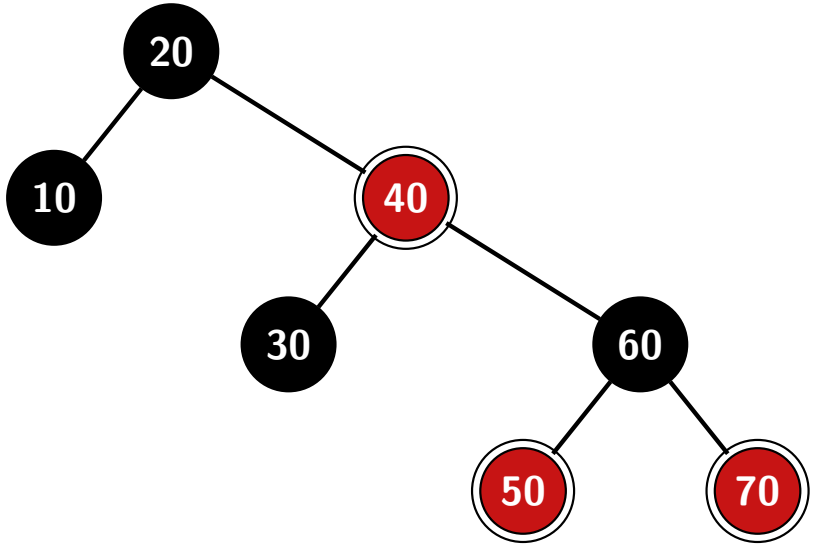


30 y 50 a negro, 40 a rojo; z sube al 40, cuyo padre 20 es negro. Termina sin rotar.

Traza: entran el 70 y el 80 I

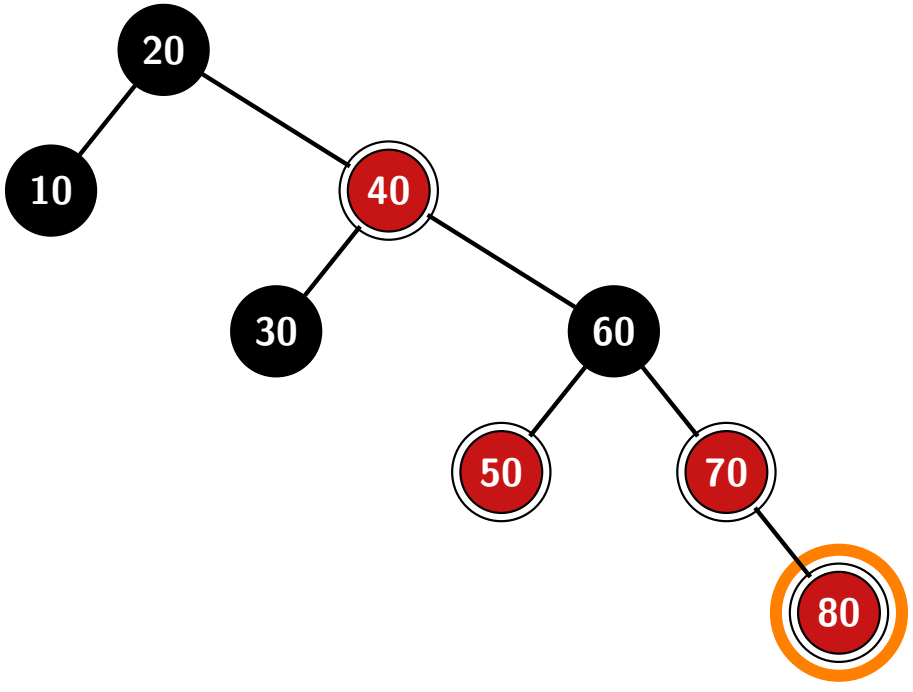


70: padre 60 rojo, tío `NIL` negro, en línea: caso 3.



60 a negro, 50 a rojo, rotar a la izquierda en el 50.

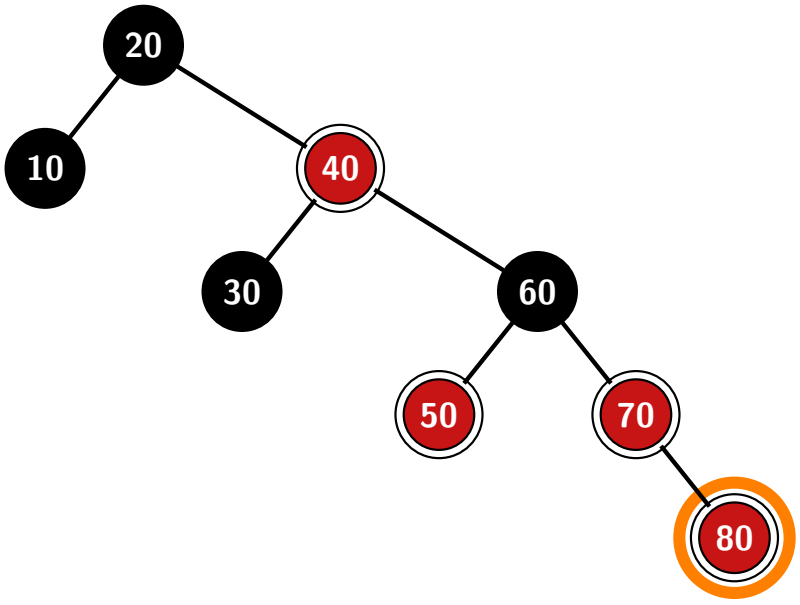
Traza: entran el 70 y el 80 II



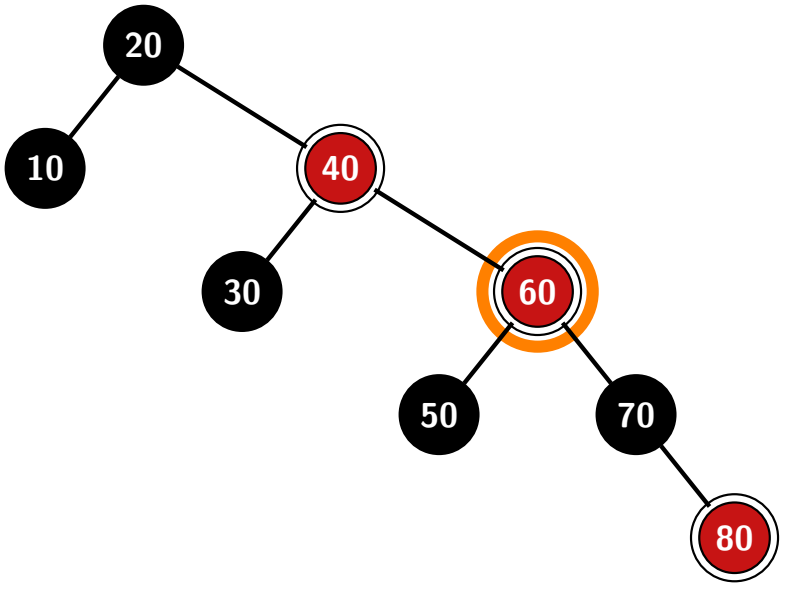
Pregunta

Entra el 80. Su padre 70 es rojo y su tío 50 también. ¿Qué pasa después de recolorear?

Traza: entran el 70 y el 80 III

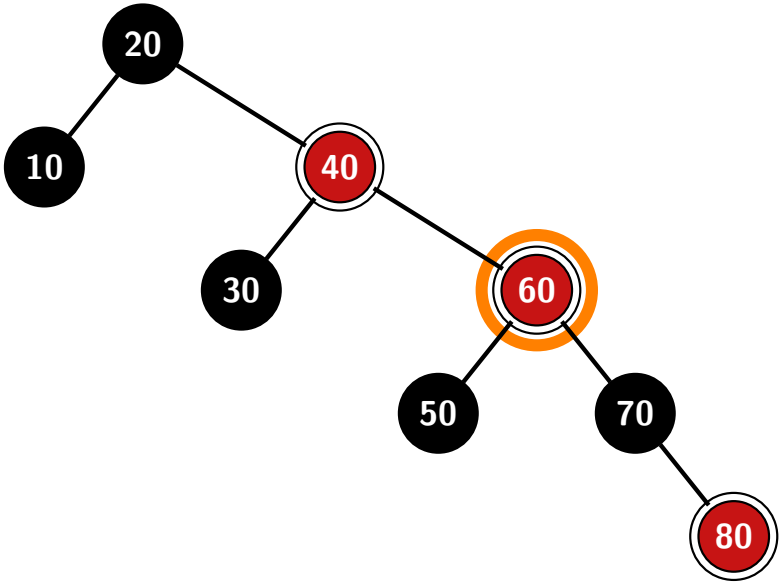


Caso 1: 50 y 70 a negro, 60 a rojo.

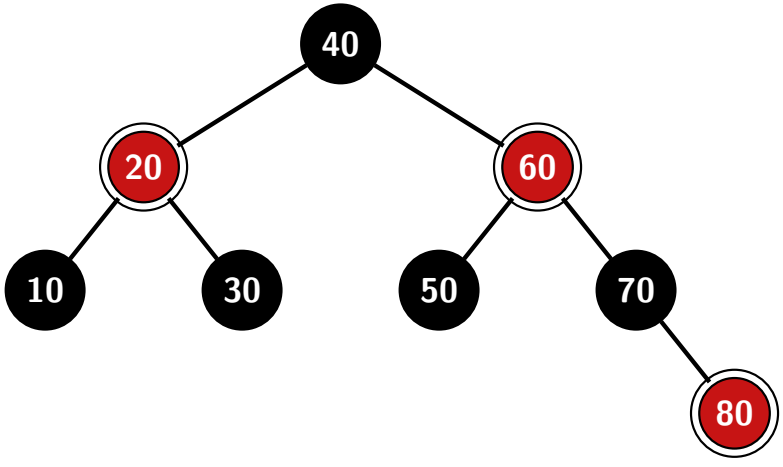


z sube al 60, cuyo padre 40 es rojo: la regla 4 se rompe más arriba.

Traza: entran el 70 y el 80 IV



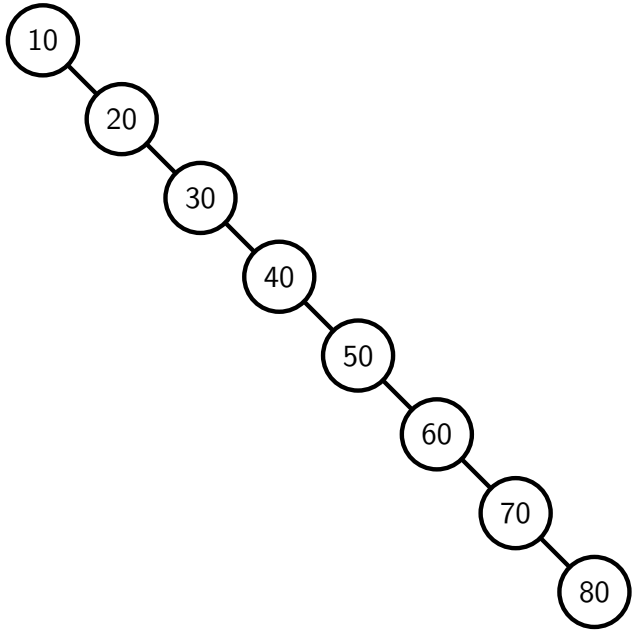
Ahora el tío de z es el 10, negro, y 20, 40 y 60 están en línea a la derecha: caso 3.



40 a negro, 20 a rojo, rotar a la izquierda en el 20. El 30 cambia de padre: era el β de la rotación.

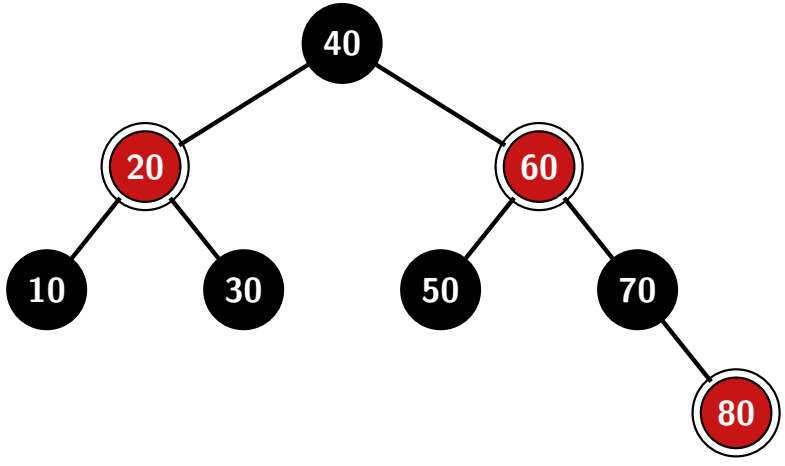
Las mismas ocho claves, dos árboles I

ABB



altura 7: buscar el 80 compara 8 veces.

rojinegro



altura 3: buscar el 80 compara 4 veces.

Las mismas ocho claves, dos árboles II

En toda la traza hubo cuatro rotaciones en ocho inserciones, una como máximo en cada una. Siguiendo la misma inserción de CLRS con `1`, `2`, `...`, `n` en orden:

n	altura del ABB	altura del rojinegro	$2\log_2(n+1)$
1 000	999	16	19,9
32 000	31 999	26	29,9
1 000 000	999 999	36	39,9

Lo que cuesta insertar I

Paso	Tiempo	Espacio aparte	Por qué
bajar hasta el `NIL`	$O(\log n)$	$O(1)$	un camino de largo $h \leq 2\log_2(n+1)$
recolorear (caso 1)	$O(\log n)$	$O(1)$	cada vez z sube dos niveles; no pasa de la raíz
rotaciones (casos 2 y 3)	$O(1)$	$O(1)$	a lo sumo dos, y después del caso 3 se termina
insertar, en total	$O(\log n)$	$O(1)$	la suma de los tres

El espacio aparte es $O(1)$ porque la versión de CLRS sube con un puntero al padre en cada nodo, sin recursión. La estructura ocupa $\Theta(n)$: cada nodo guarda la clave, tres punteros y el color.

La reparación nunca baja

Los recoloreos y las rotaciones ocurren en el camino que la búsqueda ya recorrió, de z hacia la raíz. El resto del árbol no se toca.

Borrar en un rojinegro (CLRS, §13.4) I

- Se borra como en el ABB: hoja, un hijo o dos hijos con el sucesor.
- Quitar un nodo rojo no cambia ninguna altura negra.
- Quitar uno negro deja a los caminos que pasaban por él con un negro de menos, y se rompe la regla 5. Se repara mirando al *hermano* del nodo que ocupó su lugar, en cuatro casos con recoloreos y rotaciones.
- Son las mismas herramientas de la inserción, con más casos. Se hacen a lo sumo tres rotaciones, y borrar cuesta $O(\log n)$.

Operación	Tiempo	Rotaciones	Por qué
buscar, mínimo, máximo	$O(\log n)$	0	un camino de largo $O(\log n)$
insertar	$O(\log n)$	a lo sumo 2	bajar y reparar hacia arriba
borrar	$O(\log n)$	a lo sumo 3	bajar y reparar hacia arriba
recorrer en inorden	$\Theta(n)$	0	cada nodo una vez

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

map y set son un rojinegro I

En la biblioteca de g++ (libstdc++), map y set guardan sus claves en la misma clase, _Rb_tree. El archivo bits/stl_tree.h la presenta así:

```
// Red-black tree class, designed for use in implementing STL
// associative containers (set, multiset, map, and multimap). The
// insertion and deletion algorithms are based on those in Cormen,
// Leiserson, and Rivest, Introduction to Algorithms (MIT Press,
// 1990), except that
//
// (1) the header cell is maintained with links not only to the root
// but also to the leftmost node of the tree, to enable constant
// time begin(), ...
```

`map` y `set` son un rojinegro II

De ahí salen los costos que se usaron con `map`:

- `m[k]`, `find`, `count` y `erase` cuestan $O(\log n)$ también en el peor caso: son buscar, insertar y borrar en un rojinegro.
- El recorrido con el iterador sale en orden de clave porque *es* el inorden del árbol. Avanzar el iterador va al sucesor: baja al mínimo del subárbol derecho o sube hasta el primer antepasado por la izquierda.
- `begin()` cuesta $\Theta(1)$ porque el encabezado guarda un enlace al nodo de más a la izquierda, el menor.
- En el `map`, el nodo guarda la pareja; el árbol se ordena por la clave.

`map` y `set` son un rojinegro III

`medir_set.cpp` repite el experimento de los pedidos con un `set`. Del `set` solo se mide el tiempo: la biblioteca no deja ver su árbol.

n	altura del ABB	segundos ABB	segundos `set`
1 000	999	0,0008	0,0003
4 000	3 999	0,0128	0,0016
16 000	15 999	0,2021	0,0069
32 000	31 999	0,7989	0,0142

- El `set` dobla su tiempo cuando n se dobla: n inserciones de $O(\log n)$. El ABB lo multiplica por 4.
- Con 32 000 pedidos, el `set` es 56 veces más rápido. Un millón de pedidos en orden le toman 0,58 segundos.

Tres estructuras para las mismas claves I

	ABB sin balancear	`map`, `set` (rojinegro)	`unordered_map` (tabla hash)
buscar, insertar, borrar	$O(h)$: de $\Theta(\log n)$ a $\Theta(n)$	$O(\log n)$, también en el peor caso	$O(1)$ en promedio; $\Theta(n)$ en el peor caso
recorrer en orden de clave	$\Theta(n)$	$\Theta(n)$	copiar y ordenar: $\Theta(n \log n)$
la menor clave	$O(h)$	$\Theta(1)$ con `begin()`	$\Theta(n)$, revisando todas
la primera clave $\geq x$	$O(h)$	$O(\log n)$ con `lower_bound`	no la tiene
lo que pide de la clave	`<`	`<`	una función hash y `==`

El ABB del curso sirve para entender lo que hay dentro, pero no para datos que pueden llegar en orden. El rojinegro cobra un color por nodo y unas pocas rotaciones a cambio de la garantía.

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Solo las claves I

Pregunta

Una lista de asistencia trae el código de cada estudiante cada vez que entra al aula: `42 17 88 17 5 63 42 30`. ¿Quiénes vinieron, de menor a mayor y sin repetir? Con un `map`, ¿qué valor se guardaría?

Solo las claves II

Respuesta

Vinieron `5 17 30 42 63 88`. Con un `map` habría que inventar un valor que nadie consulta. Lo que hace falta es un conjunto: claves sin repetir, en orden, sin nada al lado.

El contenedor `set`

`set<T>` guarda claves distintas de tipo `T`, ordenadas con `<`. Por dentro es el mismo árbol rojinegro del `map`, con un nodo por clave y sin valor. Se incluye con `#include <set>`.

set en C++ I

uso_set.cpp mete la lista de asistencia en un set, la imprime, la consulta y pregunta por sus bordes.

```cpp
int main() {
  set<int> s;
  int llegan[8] = {42, 17, 88, 17, 5, 63, 42, 30};
  int i = 0;
  while (i < 8) {
    s.insert(llegan[i]);      // los segundos 17 y 42 no entran
    i = i + 1;
  }
  imprimir(s);
  consultar(s);
  imprimir(s);
  bordes(s);
  return 0;
}
```

set en C++ II

```cpp
// recorre el set con su iterador: las claves salen de menor a mayor
void imprimir(set<int> &s) {
  set<int>::iterator it = s.begin();
  while (it != s.end()) {
    printf(" %d", *it);
    ++it;
  }
  printf("   size = %d\n", (int) s.size());
}
```

```cpp
// count, find y erase sobre la lista de asistencia
void consultar(set<int> &s) {
  printf("count(17) = %d, count(18) = %d\n", (int) s.count(17),
         (int) s.count(18));
  printf("find(63) == end(): %d\n", (int) (s.find(63) == s.end()));
  printf("erase(17) = %d\n", (int) s.erase(17));
  printf("erase(17) = %d\n", (int) s.erase(17));
}
```

set en C++ III

```
// la menor, la mayor y las primeras claves >= x y > x
void bordes(set<int> &s) {
  printf("menor: %d\n", *s.begin());
  set<int>::iterator ult = s.end();
  --ult;                        // end() queda despues del mayor
  printf("mayor: %d\n", *ult);
  printf("lower_bound(42) = %d\n", *s.lower_bound(42));
  printf("upper_bound(42) = %d\n", *s.upper_bound(42));
  printf("lower_bound(50) = %d\n", *s.lower_bound(50));
  printf("lower_bound(90) == end(): %d\n",
         (int) (s.lower_bound(90) == s.end()));
}
```

`set` en C++ IV

Pregunta

Antes de ver la salida: ¿qué imprime el primer `imprimir`? ¿Cuánto vale `size()`? ¿Qué devuelve el segundo `erase(17)`?

set en C++ V

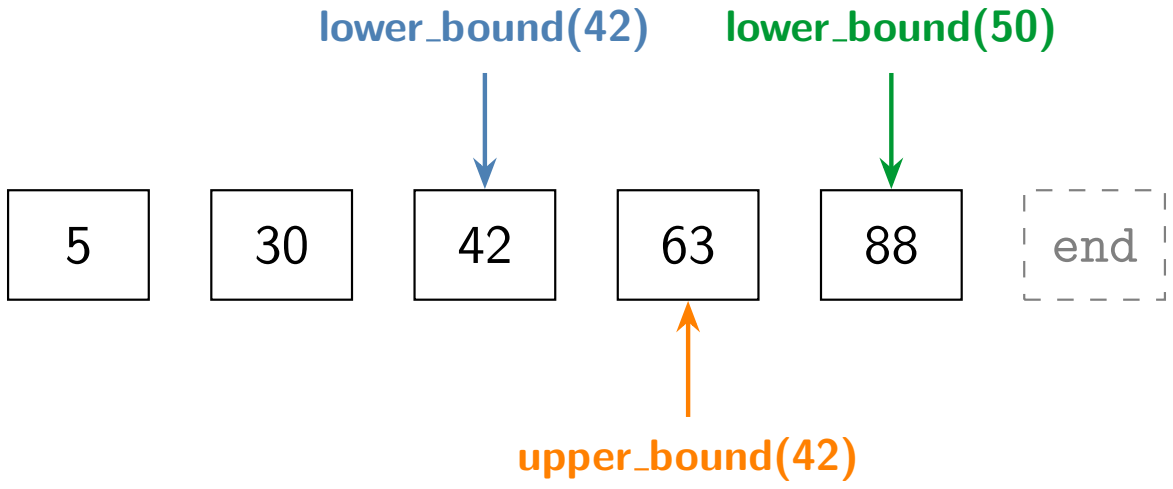
```
 5 17 30 42 63 88   size = 6
count(17) = 1, count(18) = 0
find(63) == end(): 0
erase(17) = 1
erase(17) = 0
 5 30 42 63 88   size = 5
menor: 5
mayor: 88
lower_bound(42) = 42
upper_bound(42) = 63
lower_bound(50) = 63
lower_bound(90) == end(): 1
```

`set` en C++ VI

- El segundo 17 y el segundo 42 no entran: `insert` busca la clave y, si ya está, no hace nada.
- `count` devuelve 0 o 1, y `erase(k)` devuelve cuántas claves quitó: 1 la primera vez, 0 la segunda.
- `end()` queda después del mayor. Retrocederlo una vez da el mayor.

lower_bound y upper_bound I

Con `s = {5, 30, 42, 63, 88}`:



- `lower_bound(x)`: iterador a la primera clave $\geq x$.
- `upper_bound(x)`: iterador a la primera clave $> x$.
- Si no hay ninguna, devuelven `end()`: `lower_bound(90)` es `end()`.

`lower_bound` y `upper_bound` II

Es el `lower_bound` de `<algorithm>`, pero en $O(\log n)$ bajando por el árbol, sin un arreglo ordenado. Entre los dos queda el rango de claves que están en $[a, b]$.

No sirve el de `<algorithm>`

`lower_bound(s.begin(), s.end(), x)` compila, pero sobre un `set` avanza el iterador de uno en uno: $\Theta(n)$. Se usa el del contenedor, `s.lower_bound(x)`.

Lo que cuesta cada operación de `set` I

Operación	Tiempo	Por qué
`insert(k)`	$O(\log n)$	se baja hasta el `NIL`; a lo sumo dos rotaciones
`erase(k)`	$O(\log n)$	se busca y se borra; a lo sumo tres rotaciones
`find(k)`, `count(k)`	$O(\log n)$	un camino desde la raíz
`lower_bound`, `upper_bound`	$O(\log n)$	un camino desde la raíz, guardando el mejor candidato
`size()`, `empty()`	$\Theta(1)$	el `set` lleva la cuenta
`begin()`	$\Theta(1)$	el encabezado apunta al menor
`++it`, `--it`	$O(\log n)$	ir al sucesor puede subir o bajar toda la altura
recorrido completo	$\Theta(n)$	cada arista se baja una vez y se sube una vez
`erase(desde, hasta)` de m claves	$O(\log n + m)$	se quitan de a una, sin volver a buscar
destructor	$\Theta(n)$	se libera un nodo por clave
espacio de la estructura	$\Theta(n)$	un nodo por clave, con tres punteros y el color

Los distintos, de tres maneras I

`distintos.cpp` imprime los códigos distintos de `31 12 50 12 7 31 31 64 7 50`, de menor a mayor.

```
// cada insert busca su lugar en el arbol: O(log d), d los distintos
void conSet() {
  set<int> s;
  int i = 0;
  while (i < N) {
    s.insert(codigos[i]);
    i = i + 1;
  }
  printf("set:           ");
  set<int>::iterator it = s.begin();
  while (it != s.end()) {
    printf(" %d", *it);
    ++it;
  }
  printf("   (%d distintos)\n", (int) s.size());
}
```

Los distintos, de tres maneras II

```cpp
// se copian los n, se ordenan y unique deja al frente los distintos
void conVector() {
  vector<int> v(codigos, codigos + N);
  sort(v.begin(), v.end());
  vector<int>::iterator fin = unique(v.begin(), v.end());
  v.erase(fin, v.end());
  printf("sort + unique:");
  int i = 0;
  while (i < (int) v.size()) {
    printf(" %d", v[i]);
    i = i + 1;
  }
  printf("   (%d distintos)\n", (int) v.size());
}
```

`unique` deja al frente un representante de cada grupo de iguales *seguidos* y devuelve dónde terminan; por eso va después de `sort`. El `erase` del `vector` corta lo que sobra.

Los distintos, de tres maneras III

```
// el mismo arbol que el set, con un valor que nadie consulta
void conMap() {
  map<int, bool> m;
  int i = 0;
  while (i < N) {
    m[codigos[i]] = true;
    i = i + 1;
  }
  printf("map<int,bool>:");
  map<int, bool>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("   (%d distintos)\n", (int) m.size());
}
```

```
set:            7 12 31 50 64   (5 distintos)
sort + unique: 7 12 31 50 64   (5 distintos)
map<int,bool>: 7 12 31 50 64   (5 distintos)
```

Los distintos, de tres maneras IV

Con n códigos y d distintos:

Forma	Tiempo	Espacio	Por qué
`set`	$O(n \log d)$	$\Theta(d)$	n inserciones en un árbol de a lo sumo d claves
`sort` + `unique`	$\Theta(n \log n)$	$\Theta(n)$	copia los n, ordena y recorre una vez
`map<int, bool>`	$O(n \log d)$	$\Theta(d)$	el mismo árbol, con un `bool` de más en cada nodo

- Si todos los códigos llegan juntos y no cambian, `sort` y `unique` bastan.
- Si llegan de a uno y entre llegada y llegada se pregunta si un código ya está, el `set` responde en $O(\log d)$ sin reordenar nada.

¿set, map o unordered_map? I

Lo que se guarda y se pregunta	Contenedor
solo claves; si están, en orden, la menor, la siguiente a x	`set`
una clave con un valor, y el orden de las claves aparece en la salida o se pregunta por rangos	`map`
una clave con un valor, y solo se busca, se suma o se cuenta por clave	`unordered_map`
cuántas veces aparece cada clave	`map<T, int>` o `unordered_map<T, int>`, según haga falta el orden

¿set, map o unordered_map? II

El set no guarda repetidos

Con `3 3 7`, el `set` queda con `3 7` y `size()` vale 2. Si las repeticiones importan, cada clave lleva su cuenta en un `map`.

Una clave del set no se modifica

`*it` se puede leer pero no asignar: cambiar la clave en su sitio rompería el orden del árbol. Se borra la vieja y se inserta la nueva, $O(\log n)$ cada una.

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

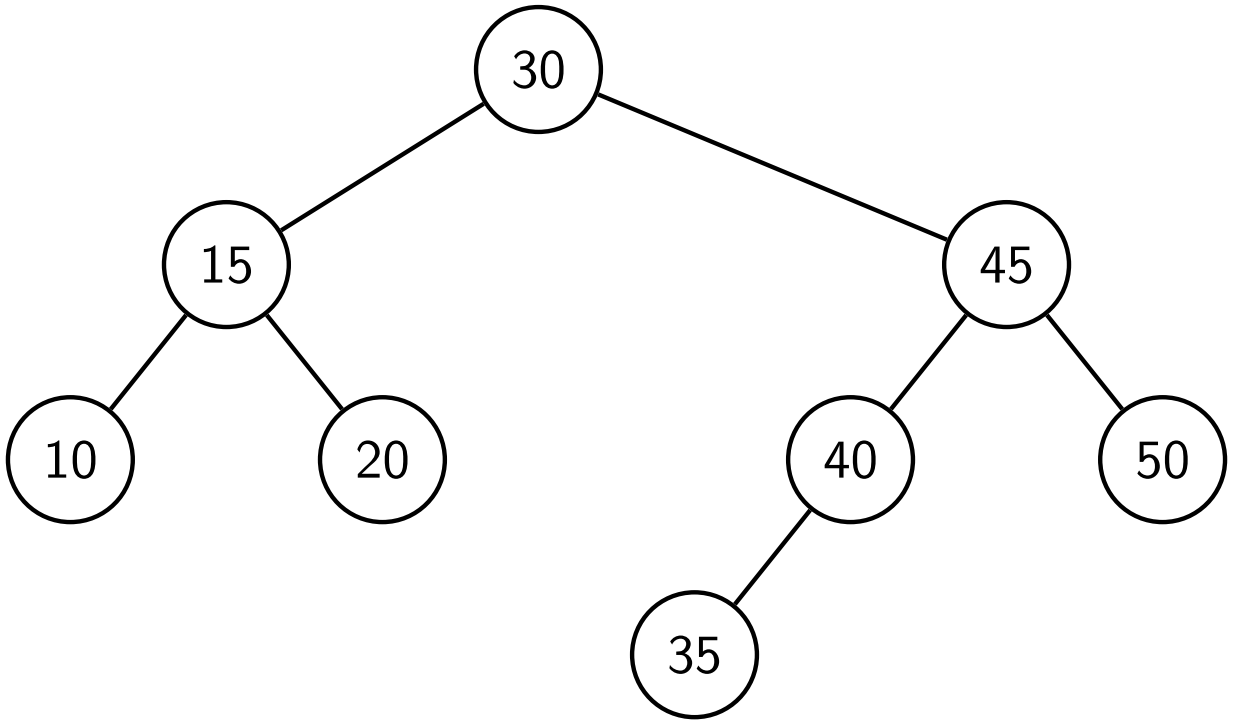
8 Para el juez

9 Cierre

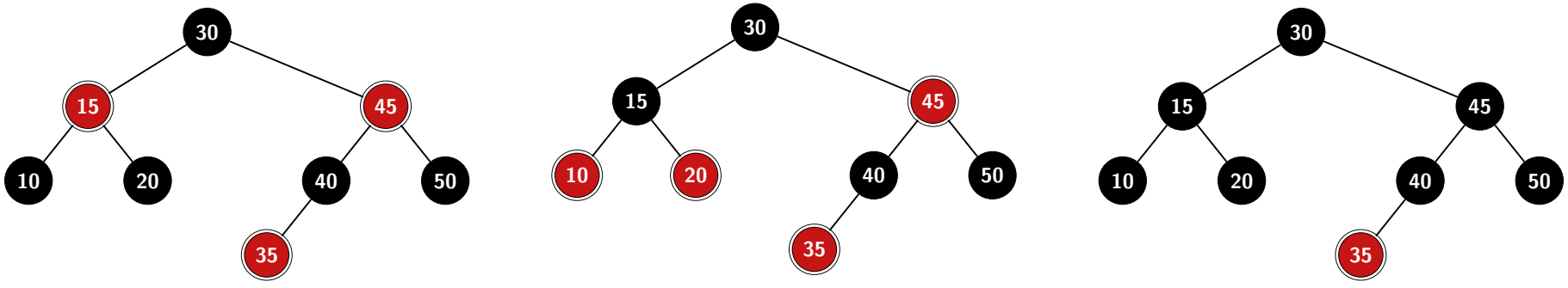
Ejercicio en parejas: colorear I

Pregunta

Pinten cada nodo de rojo o de negro para que el árbol sea rojinegro. ¿Hay una sola forma? ¿Qué nodo queda del mismo color en todas?



Ejercicio en parejas: colorear II



Respuesta

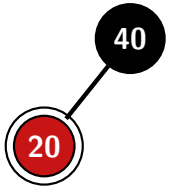
Hay tres formas: 15 y 45 rojos; 10, 20 y 45 rojos; o solo el 35 rojo. En las tres, 30, 40 y 50 son negros y el 35 es rojo: el camino 30 45 40 35 es el más largo y necesita un rojo, y el 40 no puede serlo si su hijo 35 ya lo es.

Ejercicio en parejas: insertar I

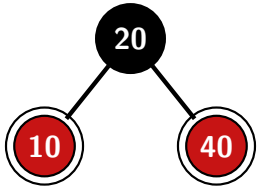
Pregunta

En un rojinegro vacío inserten 40 20 10 30 35. Dibujen el árbol después de cada inserción y digan qué caso se usó.

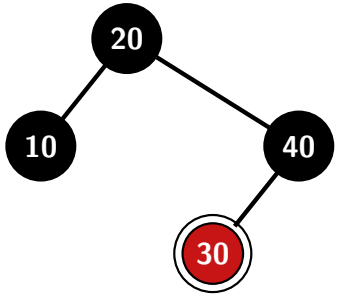
Ejercicio en parejas: insertar II



40 y 20: sin arreglo.

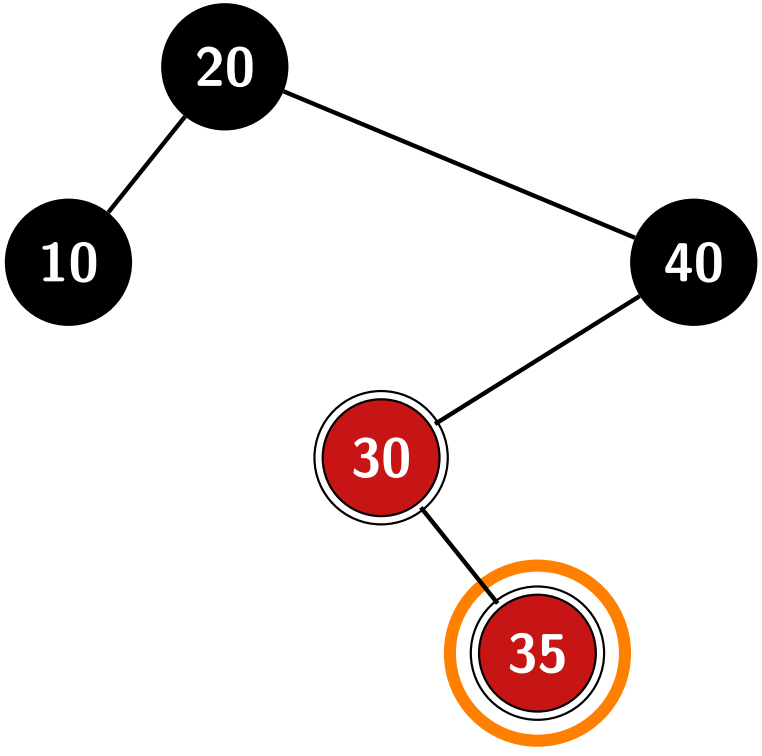


10: tío `NIL`, en línea a la izquierda: caso 3, rotar a la derecha en el 40.

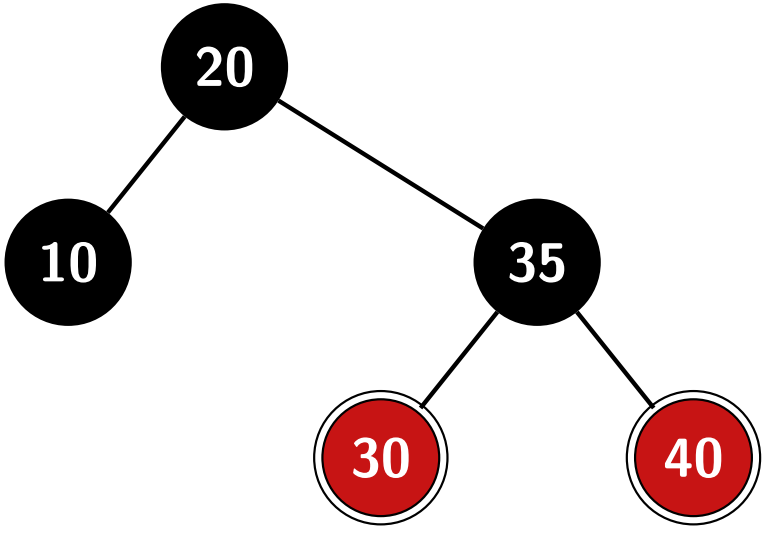


30: tío 10 rojo: caso 1. El 20 queda rojo en la raíz y se pinta de negro.

Ejercicio en parejas: insertar III



35: padre 30 rojo, tío `NIL`. El 35 es hijo derecho del 30 y el 30 es hijo izquierdo del 40: en ángulo, caso 2.



Rotar a la izquierda en el 30 y después, como caso 3, 35 a negro, 40 a rojo y rotar a la derecha en el 40.

Ejercicio en parejas: contar en un rango I

Pregunta

Con un `set<int>` y sin recorrerlo entero, cuenten cuántas claves hay en $[a, b]$. Con `s = {4, 9, 15, 22, 30, 41, 57}`, ¿cuántas hay en $[10, 40]$ y en $[42, 56]$?

Ejercicio en parejas: contar en un rango II

```cpp
// Claves en [a, b]: se baja hasta a y se camina hasta pasar b.
int contarRango(set<int> &s, int a, int b) {
  int cuenta = 0;
  set<int>::iterator it = s.lower_bound(a);
  set<int>::iterator fin = s.upper_bound(b);
  while (it != fin) {
    cuenta = cuenta + 1;
    ++it;
  }
  return cuenta;
}
```

En $[10, 40]$ hay 3: 15, 22 y 30. En $[42, 56]$, ninguna: los dos iteradores apuntan al 57 y el ciclo no da ninguna vuelta. Cuesta $O(\log n + m)$, con m las claves del rango: dos bajadas por el árbol y m avances.

Ejercicio en parejas: el más cercano I

Pregunta

Escriban `masCercano(s, x)`: la clave del `set` más cercana a x; si hay dos a la misma distancia, la menor. Con el mismo `s`, ¿qué da para 26, 12, 1 y 99?

Ejercicio en parejas: el más cercano II

```
// La clave mas cercana a x; si empatan dos, la menor. s no vacio.
int masCercano(set<int> &s, int x) {
  assert(!s.empty());
  set<int>::iterator techo = s.lower_bound(x);   // la primera >= x
  set<int>::iterator piso = techo;               // la de antes
  int res;
  if (techo == s.begin()) {
    res = *techo;
  } else {
    --piso;
    if (techo == s.end() || x - *piso <= *techo - x) {
      res = *piso;
    } else {
      res = *techo;
    }
  }
  return res;
}
```

Ejercicio en parejas: el más cercano III

La más cercana es `lower_bound(x)`, la primera $\geq x$, o la que está justo antes. Se revisan los dos bordes: si la primera $\geq x$ es la menor de todas, no hay anterior; si no hay ninguna $\geq x$, gana la anterior, que es la mayor. El `||` deja de evaluar en cuanto `techo` es `end()`, antes de leer `*techo`.

```
masCercano(26) = 22
masCercano(12) = 9
masCercano(1) = 4
masCercano(99) = 57
```

Para 26, el 22 y el 30 quedan a 4 y gana el menor. $O(\log n)$: una bajada y un retroceso del iterador.

Propuestos para la casa I

1. **Seguir la traza.** Continúen la traza de los pedidos con 90 y 100. ¿Cuántas rotaciones pide cada uno? ¿Qué altura queda?
2. **De mayor a menor.** Inserten 80 70 60 50 40 30 20 10 en un rojinegro vacío y comparen con la traza creciente.
3. **¿Rojinegro o no?** Dibujen un ABB de cuatro nodos que no se pueda colorear como rojinegro, y digan por qué.
4. **Altura negra.** ¿Cuál es el rojinegro con menos nodos cuya raíz tiene altura negra 3? ¿Y el que tiene más nodos?

Propuestos para la casa II

5. **Borrar los menores.** Con un `set`, quiten de una vez todas las claves menores que x, sin un ciclo.
6. **Solo en uno.** Dadas dos listas de códigos, impriman en orden los que están en una sola de las dos.
7. **Los k menores.** Impriman los k valores distintos más pequeños de una lista. El ciclo se detiene por su condición.
8. **Siguiente libre.** Un `set` guarda los casilleros ocupados. Para un x dado, impriman el menor casillero libre $\geq x$.

Pistas para los propuestos I

1. El 90 cae en el caso 3, con una rotación. El 100 cae dos veces seguidas en el caso 1, que sube hasta la raíz sin rotar. La altura queda en 4.
2. Es la misma traza reflejada: cada rotación a la izquierda se vuelve a la derecha. La forma final es el espejo.
3. Con cuatro nodos en lista no hay forma: el camino más largo tiene 4 nodos y el más corto, 1, y $4 > 2 \cdot 1$ sin importar los colores.
4. Con altura negra 3 desde la raíz, el más chico es el árbol perfecto todo negro de 7 nodos. El más grande alterna niveles negros y rojos bajo una raíz negra: seis niveles llenos, 63 nodos.

Pistas para los propuestos II

5. `erase(desde, hasta)` con `begin()` y `lower_bound(x)`: $O(\log n + m)$.
6. Se recorren los dos `set` y se imprime cada clave cuyo `count` en el otro es 0, o se cuenta en un `map` cuántas listas la traen. $O((n_1 + n_2) \log n)$.
7. Se insertan todos en un `set` y se recorre con un contador; la condición del `while` pide que el iterador no sea `end()` y que el contador no llegue a k.
8. Se empieza en x y se avanza mientras el casillero esté. Con `lower_bound(x)` y el iterador se camina por las claves seguidas $x, x+1, \ldots$ sin volver a buscar.

Ejercicio de cierre I

Pregunta

Construyan, con la menor cantidad de nodos posible, un rojinegro donde un camino de la raíz a un `NIL` tenga el doble de nodos internos que otro. ¿Qué secuencia de inserciones lo produce?

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Para el juez: UVa 10815 I

UVa 10815, Andy's First Dictionary (UVa Online Judge)
Enunciado: `https://onlinejudge.org/external/108/10815.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1756`

Qué pide

Llega un texto de a lo sumo 5 000 líneas, de hasta 200 caracteres cada una, hasta el fin de la entrada. Una palabra es una racha seguida de letras, mayúsculas o minúsculas, y `Apple`, `apple` y `APPLE` son la misma. Se imprimen las palabras distintas en minúscula, una por línea, en orden alfabético. Hay a lo sumo 5 000 distintas.

Para el juez: UVa 10815 II

¿Entendimos el problema?

La línea `road. The sign read: "Disneyland Left."` aporta `road`, `the`, `sign`, `read`, `disneyland` y `left`. ¿Por qué `Disneyland`, que aparece tres veces en el ejemplo, sale una sola vez?

Cómo atacarlo

Un `set<string>`: cada palabra, ya en minúscula, se inserta, y las repetidas no entran. El recorrido final sale en orden alfabético, porque `string` se compara letra por letra. Con W palabras en total, d distintas y L letras por palabra, cuesta $O(W \cdot L \log d)$: cada comparación de dos palabras mira hasta L letras.

Para el juez: UVa 10815 III

Las trampas

- `scanf("%s")` lee trozos separados por espacios, no palabras: `Left."` trae un punto y unas comillas, y `don't` son dos palabras, `don` y `t`. Cada trozo se parte en sus rachas de letras.
- Los dígitos y los guiones también separan: `stop-me` da `stop` y `me`.
- Las mayúsculas se pasan a minúscula antes de insertar; si no, `The` y `the` salen dos veces, y en desorden, porque las mayúsculas van antes que todas las minúsculas.
- Un trozo puede medir 200 caracteres: el arreglo lleva 201.

Para el juez: UVa 11849 I

UVa 11849, CD (UVa Online Judge)
Enunciado: `https://onlinejudge.org/external/118/11849.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2949`

Qué pide

Jack tiene N discos y Jill M, con $N, M \leq 10^6$, cada uno a lo sumo una copia de cada título. Llegan los números de catálogo de Jack en orden creciente y después los de Jill, también en orden, enteros positivos hasta 10^9. Se imprime cuántos títulos tienen los dos. La entrada trae varios casos y termina con `0 0`.

Para el juez: UVa 11849 II

¿Entendimos el problema?

Jack tiene `1 2 3` y Jill `1 2 4`: los dos tienen el 1 y el 2, y la salida es 2. ¿Qué imprime un caso con `N = 0`?

Cómo atacarlo

Los de Jack entran a un `set<int>`, y cada uno de Jill suma su `count`. Cuesta $O((N+M)\log N)$ y $\Theta(N)$ de espacio. Con 10^6 y 10^6 tarda 1,19 segundos sin optimizar, en el equipo donde se preparó la sesión.

Para el juez: UVa 11849 III

Las trampas

- N y M pueden ser 0 por separado. La entrada termina solo cuando los dos lo son: `while (n != 0 || m != 0)`. Un caso `0 2` imprime 0 y no es el final.
- Hasta dos millones de números por caso: se leen con `scanf`.
- Las dos listas llegan ordenadas. Dos índices que avanzan a la vez, como al mezclar, cuentan los comunes en $\Theta(N + M)$ y sin guardar nada: el `set` funciona, pero no aprovecha ese orden.

Contenido I

1 El problema: claves que llegan en orden

2 Las cinco reglas

3 Rotaciones

4 Insertar y borrar

5 Lo que hay dentro de `map`

6 El contenedor `set`

7 Ejercicios

8 Para el juez

9 Cierre

Resumen I

- Un ABB que recibe claves en orden degenera en lista, y reconstruirlo balanceado tras cada inserción cuesta $\Theta(n)$.
- Un árbol rojinegro pinta cada nodo de rojo o de negro con cinco reglas: raíz y hojas `NIL` negras, ningún rojo con hijo rojo, y la misma cantidad de negros en todo camino hacia abajo, y así $h \leq 2\log_2(n+1)$.
- Una rotación sube un nodo cambiando tres enlaces, en $O(1)$, y conserva el inorden.
- Insertar baja como en el ABB, pinta de rojo y repara hacia arriba según el tío: rojo, recolorear y subir; negro, una o dos rotaciones. Borrar usa las mismas herramientas. Todo en $O(\log n)$.
- `map` y `set` son un rojinegro: por eso cuestan $O(\log n)$ y su recorrido es el inorden.
- `set` guarda claves distintas en orden, y `lower_bound` y `upper_bound` encuentran la primera clave $\geq x$ o $> x$ en $O(\log n)$.

Resumen II

Lo que sigue

Los montículos (CLRS, cap. 6) y la cola de prioridad: sacar siempre el mayor, o el menor, en $O(\log n)$ con un árbol guardado en un arreglo.

Referencias I


T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 13, árboles rojinegros: §13.1, propiedades y lema 13.1; §13.2, rotaciones; §13.3, inserción; §13.4, borrado.


R. Thareja. *Data Structures Using C*, 2.ª ed. Oxford University Press, 2014. §10.5, *Red-Black Trees*, pp. 327–337; §10.5.1, propiedades, p. 328.


GNU Project. *The GNU C++ Library* (libstdc++), archivo `bits/stl_tree.h`, clase `_Rb_tree`.


UVa Online Judge. Problema 10815, *Andy's First Dictionary*: `https://onlinejudge.org/external/108/10815.pdf`.


UVa Online Judge. Problema 11849, *CD*: `https://onlinejudge.org/external/118/11849.pdf`.